Interactive AI Museums Implementation Plan

> ***For agentic workers:*** *REQUIRED SUB-SKILL: Use* `superpowers:subagent-driven-development` *(recommended) or* `superpowers:executing-plans` *to implement this plan task-by-task. Steps use checkbox (* `- [ ]` *) syntax for tracking.*

Goal: Ship a 6-week, browser-based interactive AI-generated museum where the user types a prompt and walks through a 3D exhibit on rails, picking up and dissecting AI-generated objects with internal structure.

Architecture: FastAPI backend (Modal `@asgi_app`) + Vite/React/TS frontend + Modal-hosted ML (HunyuanWorld for scenes, FLUX.1-schnell → TRELLIS.2 for objects) + Claude API orchestrator. No queue, no DB; per-exhibit `manifest.json` on Modal Volume is the single source of truth. Camera-on-rails locomotion through the AI scene; stencil-based capped clipping for dissection.

Tech Stack: Python 3.12, FastAPI, Pydantic v2, Modal, Anthropic SDK, TypeScript 5, Vite 5, React 19, three.js (no R3F), Zustand, Tailwind, Catmull-Rom splines, WebGL stencil buffer, GLB+PBR.

Source spec: `docs/superpowers/specs/2026-05-14-interactive-museums-design.md`

File Structure (decomposition lock-in)

Every file's responsibility, before defining tasks. Files that change together live together. Each file does one thing.

```
/mnt/3D_Mapping/
├── README.md                                          project blurb + quickstart
├── pyproject.toml                                     uv workspace root
├── Makefile                                           make dev | smoke | spike |
deploy
├── .gitignore
├── .env.example                                       placeholder keys
├── docs/
│   ├── superpowers/specs/2026-05-14-interactive-museums-design.md
│   └── superpowers/plans/2026-05-14-interactive-museums-plan.md  (this file)
├── SPIKE_RESULT.md                                    populated end of Week 1
│
├── backend/
│   ├── Dockerfile
│   ├── pyproject.toml
│   └── app/
│       ├── __init__.py
│       ├── main.py                                    FastAPI app + Modal asgi_app
│       ├── config.py                                  pydantic-settings (env vars)
│       ├── schemas.py                                 Manifest, Object, Scene,
Spline, Pedestal, Station
│       ├── routing.py                                 stage retry policy + fallback
orchestration
│       ├── api/
│       │   ├── __init__.py
│       │   ├── exhibits.py                            POST /api/exhibits, GET
/api/exhibits/{id}, SSE
```

```
│       │   └── health.py                          GET /health
│       ├── orchestrator/
│       │   ├── __init__.py
│       │   ├── claude.py                          Anthropic SDK client + retry-
with-feedback
│       │   ├── prompts.py                         versioned system prompt + 3
few-shot examples
│       │   └── layout_templates.py                linear, hub-and-spokes,
semicircle (helpers)
│       ├── modal_client/
│       │   ├── __init__.py
│       │   ├── scene.py                           scene_gen wrapper (HunyuanWorld
or stock)
│       │   ├── object.py                          object_gen wrapper (FLUX →
TRELLIS.2)
│       │   └── _modal.py                          Modal app handle + image defs
(shared)
│       ├── stock_rooms/
│       │   ├── manifest.json                      metadata for 10 stock rooms
│       │   └── rooms/                             *.glb files (gitignored, ~5MB
each)
│       └── storage/
│           ├── __init__.py
│           └── volume.py                          read/write per-exhibit dir on
Modal Volume
│
├── modal/                                         Modal function definitions (run
via modal CLI)
│   ├── shared.py                                  app, volume, image base
│   ├── scene_gen.py                               @app.function for HunyuanWorld
+ stock loader
│   └── object_gen.py                              @app.function for FLUX →
TRELLIS.2 chain
│
├── frontend/
│   ├── Dockerfile
│   ├── package.json
│   ├── vite.config.ts
│   ├── tsconfig.json
│   ├── index.html
│   ├── tailwind.config.js
│   ├── postcss.config.js
│   └── src/
│       ├── main.tsx
│       ├── App.tsx                                router + global layout
│       ├── api/
│       │   ├── client.ts                          typed wrappers around backend
│       │   └── types.ts                           manifest types (mirror of
backend schemas)
│       ├── pages/
│       │   ├── Landing.tsx                        hero + curated gallery + CTA
│       │   ├── Generate.tsx                       prompt input + SSE progress
```

```
│       │   └── Exhibit.tsx                              three.js viewer host
│       ├── viewer/
│       │   ├── ExhibitViewer.tsx                        three.js root component
│       │   ├── Scene.tsx                                scene GLB loader + lighting
│       │   ├── Pedestals.tsx                            pedestal meshes + object
placement
│       │   ├── locomotion/
│       │   │   ├── Spline.ts                            Catmull-Rom position(t),
tangent(t)
│       │   │   ├── RailController.ts                    advance/retreat + tween + yaw-
align
│       │   │   ├── FocusController.ts                   enter/exit focus; hero-pose
calc
│       │   │   ├── MouseLook.ts                         pointer-lock + yaw/pitch deltas
│       │   │   └── ModeManager.ts                       walking ↔ focused state machine
│       │   ├── dissection/
│       │   │   ├── ClippingPlane.ts                     plane math + axis-aware offset
│       │   │   ├── CappedClipMaterial.ts                extends MeshStandardMaterial;
stencil + cap
│       │   │   ├── ExplodedView.ts                      per-mesh centroid + radial
translation
│       │   │   ├── DissectionPanel.tsx                  React side panel
│       │   │   └── DissectionStore.ts                   Zustand: focus state
│       │   ├── hud/
│       │   │   ├── StationIndicator.tsx                 top-center label
│       │   │   ├── ProgressBar.tsx                      bottom spline progress
│       │   │   └── HintChip.tsx                         fading control hints
│       │   └── store/
│       │       ├── locomotionStore.ts                   Zustand: camera mode state
│       │       └── exhibitStore.ts                      Zustand: loaded manifest
│       └── components/
│           ├── Header.tsx
│           ├── Footer.tsx
│           └── PromptCard.tsx
│
├── references/                                          gitignored, study material
│   ├── TRELLIS.2/                                       cloned in Task 7
│   ├── HunyuanWorld/                                    cloned in Task 7
│   ├── supersplat/                                      relocated from existing dir
│   ├── splatviz/                                        relocated from existing dir
│   └── (existing 3DGS clones, relocated from project root)
│
└── _archive/                                            gitignored, old files preserved
    ├── PLAN_v4.md / .pdf                                prior iterations
    ├── PLAN_FINAL.md / .pdf
    ├── PLAN_REVIEW.md / .pdf
    ├── PROJECT_REPORT.pdf
    ├── (various .mp4, .exe, .zip, .pdf — see Task 1)
    └── Papers/
```

Phase 1 — Week 1: Project init + Spike (M0)

Tasks 1–8 are deterministic scaffold. Tasks 9–12 are the spike — bounded research, not TDD.

Task 1: Cleanup root + `git init`

Files: move existing 3DGS clones into `references/`, misc artifacts into `_archive/`, delete `magic-portfolio/`, create `.gitignore`.

- [ ] **Step 1: Audit project root.** Run `ls -A /mnt/3D_Mapping/`. Expect a mix of clones (`gaussian-splatting`, `gsplat`, ...), prior docs (`PLAN_v4.md`, `PROJECT_REPORT.pdf`, ...), and misc artifacts (`*.zip`, `*.exe`, `Papers/`, `.conda/`, ...).

- [ ] **Step 2: Create destination dirs.**

```
mkdir -p /mnt/3D_Mapping/references /mnt/3D_Mapping/_archive
```

- [ ] **Step 3: Move 3DGS clones into `references/`.**

```
cd /mnt/3D_Mapping
for d in 4DGaussians EasyVolcap gaussian-splatting gsplat fast-gaussian-
rasterization UnityGaussianSplatting UnityGaussianSplatting-main; do
  [ -e "$d" ] && mv "$d" references/
done
```

- [ ] **Step 4: Move loose artifacts into `_archive/`.**

```
cd /mnt/3D_Mapping
for p in \
  PLAN_v4.md PLAN_v4.pdf PLAN_REVIEW.md PLAN_REVIEW.pdf PLAN_FINAL.md PLAN_FINAL.pdf
\
  PROJECT_REPORT.pdf project_proposal.pdf 2023610_YashVerma.pdf \
  1.mp4 laptop.mp4 chair.psht test.ipynb easyvocap.txt \
  Papers \
  colmap-x64-windows-cuda colmap-x64-windows-cuda.zip \
  ffmpeg-2025-03-20-git-76f09ab647-essentials_build ffmpeg-2025-03-20-git-
76f09ab647-essentials_build.7z \
  ffmpeg-7.1.1 ffmpeg-7.1.1.tar.xz \
  ImageMagick-7.1.1-46-Q16-HDRI-x64-dll.exe Postshot-BETA-0.5.115.exe \
  UnityGaussianSplatting-main.zip zju3dv.zip \
  images data input output sparse stereo distorted \
  .conda .vscode; do
  [ -e "$p" ] && mv "$p" _archive/
done
```

- [ ] **Step 5: Delete `magic-portfolio/`** (user-authorized in v1 round).

```
rm -rf /mnt/3D_Mapping/magic-portfolio
```

- [ ] **Step 6: Verify root clean.** `ls -A /mnt/3D_Mapping/` should show only `references/`, `_archive/`, `docs/`, `spikes/` (if it exists).
- [ ] **Step 7: `git init` + `.gitignore`.**

```
cd /mnt/3D_Mapping && git init -b main
```

Write `.gitignore`:

```
node_modules/
dist/
build/
__pycache__/
*.pyc
.venv/
.pytest_cache/
.mypy_cache/
.ruff_cache/
.env
.env.*
!.env.example
*.glb
*.ply
*.ksplat
*.mp4
!docs/**/*.png
!frontend/public/**/*.png
!backend/stock_rooms/rooms/*.glb
.DS_Store
.vscode/
.idea/
references/
_archive/
data/
spikes/spike_a/*.png
spikes/spike_a/*.glb
spikes/spike_b/*.glb
spikes/spike_b/*.mp4
```

- [ ] **Step 8: Initial commit.**

```
cd /mnt/3D_Mapping
git add docs/ .gitignore
git commit -m "chore: init repo, preserve prior docs in _archive/"
```

Task 2: Backend package skeleton

Files: `backend/pyproject.toml`, `backend/app/__init__.py`, `backend/app/main.py`, `backend/app/config.py`, `backend/app/api/__init__.py`, `backend/app/api/health.py`.

- [ ] **Step 1: Write `backend/pyproject.toml`.**

```
[project]
name = "museums-backend"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
  "fastapi>=0.115",
  "uvicorn[standard]>=0.30",
  "pydantic>=2.9",
  "pydantic-settings>=2.5",
  "httpx>=0.27",
  "sse-starlette>=2.1",
  "anthropic>=0.40",
  "modal>=0.66",
  "python-multipart>=0.0.9",
  "pillow>=10.4",
]

[project.optional-dependencies]
dev = ["pytest>=8.3", "pytest-asyncio>=0.24", "ruff>=0.6"]

[tool.ruff]
line-length = 100
target-version = "py312"

[tool.pytest.ini_options]
asyncio_mode = "auto"
testpaths = ["tests"]
```

- [ ] **Step 2: Empty package init `backend/app/__init__.py`.**

```
"""Interactive AI Museums backend."""
__version__ = "0.1.0"
```

- [ ] **Step 3: `backend/app/config.py`.**

```
from functools import lru_cache
from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")
```

```
    anthropic_api_key: str = ""
    claude_model: str = "claude-sonnet-4-6"
    modal_app_name: str = "museums-ml"
    data_dir: str = "/data"
    max_objects_per_exhibit: int = 5
    min_objects_per_exhibit: int = 3
    per_ip_daily_cap: int = 5
    daily_modal_budget_usd: float = 10.0


@lru_cache
def get_settings() -> Settings:
    return Settings()
```

- [ ] **Step 4:** `backend/app/api/__init__.py` (empty file).
- [ ] **Step 5:** `backend/app/api/health.py`.

```
from fastapi import APIRouter

router = APIRouter()


@router.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}
```

- [ ] **Step 6:** `backend/app/main.py`.

```
from fastapi import FastAPI

from .api.health import router as health_router

app = FastAPI(title="museums-backend", version="0.1.0")
app.include_router(health_router)


@app.get("/")
def root() -> dict[str, str]:
    return {"name": "museums-backend", "docs": "/docs"}
```

- [ ] **Step 7: Verify boot.**

```
cd /mnt/3D_Mapping/backend
uv sync
uv run uvicorn app.main:app --port 8000 &
sleep 2
```

```
curl -fsS http://localhost:8000/health
pkill -f 'uvicorn app.main'
```

Expected: `{"status":"ok"}`.

- [ ] **Step 8: Commit.**

```
cd /mnt/3D_Mapping
git add backend/
git commit -m "feat(backend): minimal FastAPI app with /health"
```

Task 3: Manifest schemas (Pydantic + tests)

Files: `backend/app/schemas.py`, `backend/tests/__init__.py`, `backend/tests/test_schemas.py`.

- [ ] **Step 1: Write failing test `backend/tests/test_schemas.py`.**

```
"""Tests for the canonical Manifest contract."""
import pytest
from pydantic import ValidationError

from app.schemas import Manifest


def _valid_payload() -> dict:
    return {
        "schema_version": "v4.1",
        "exhibit_id": "01HZ8X9A2B0000000000000001",
        "created_at": "2026-05-14T18:00:00Z",
        "user_prompt": "A Victorian study with a pocket watch",
        "title": "The Inventor's Study",
        "description": "An evening in a 19th-century inventor's room.",
        "scene": {
            "generator": "hunyuanworld",
            "scene_prompt": "Victorian study interior, oak panelling",
            "stock_room_id": None,
            "scene_url": "/exhibits/01HZ8X9A2B0000000000000001/scene/scene.glb",
            "ambient_light": {"color": "#f4e2c4", "intensity": 0.6},
            "fog": {"color": "#1a1208", "near": 8.0, "far": 25.0},
        },
        "spline": {
            "control_points": [[0.0, 1.7, 4.0], [-1.0, 1.7, 2.5], [1.5, 1.7, -1.0]],
            "stations": [
                {"t": 0.0, "label": "entrance", "pedestal_id": None,
"preferred_yaw_deg": 0},
                {"t": 1.0, "label": "the watch", "pedestal_id": "pedestal_1",
"preferred_yaw_deg": -90},
            ],
            "travel_time_s": 6.0,
        },
```

```
        "pedestals": [
            {
                "id": "pedestal_1",
                "position": [-1.0, 0.0, 0.5],
                "height": 0.95,
                "object_id": "watch_1",
            }
        ],
        "objects": [
            {
                "id": "watch_1",
                "label": "Antique pocket watch",
                "placard_description": "Brass, c. 1880.",
                "image_prompt": "antique brass pocket watch, hero shot, white bg",
                "image_url":
"/exhibits/01HZ8X9A2B0000000000000001/objects/watch_1.png",
                "glb_url":
"/exhibits/01HZ8X9A2B0000000000000001/objects/watch_1.glb",
                "rotation_y_deg": 30,
                "scale": 0.08,
                "dissectable": True,
                "preferred_clip_axis": "y",
                "preferred_clip_t": 0.5,
            }
        ],
        "stage": "ready",
        "timing": {
            "orchestrator_ms": 1850,
            "scene_gen_ms": 240000,
            "objects_gen_ms": {"watch_1": 31200},
            "total_ms": 273050,
        },
        "errors": [],
    }


def test_valid_manifest_parses() -> None:
    m = Manifest.model_validate(_valid_payload())
    assert m.exhibit_id == "01HZ8X9A2B0000000000000001"
    assert m.scene.generator == "hunyuanworld"
    assert len(m.objects) == 1


def test_rejects_negative_travel_time() -> None:
    p = _valid_payload()
    p["spline"]["travel_time_s"] = -1.0
    with pytest.raises(ValidationError):
        Manifest.model_validate(p)


def test_rejects_spline_with_one_station() -> None:
    p = _valid_payload()
```

```
    p["spline"]["stations"] = [{"t": 0.0, "label": "lone", "pedestal_id": None,
"preferred_yaw_deg": 0}]
    with pytest.raises(ValidationError):
        Manifest.model_validate(p)


def test_rejects_unknown_stage() -> None:
    p = _valid_payload()
    p["stage"] = "not_a_stage"
    with pytest.raises(ValidationError):
        Manifest.model_validate(p)


def test_rejects_bad_clip_axis() -> None:
    p = _valid_payload()
    p["objects"][0]["preferred_clip_axis"] = "w"
    with pytest.raises(ValidationError):
        Manifest.model_validate(p)
```

- [ ] **Step 2: Run test, verify fail.**

```
cd /mnt/3D_Mapping/backend && uv run pytest tests/test_schemas.py -v
```

Expected: ImportError for `app.schemas.Manifest`.

- [ ] **Step 3: Implement `backend/app/schemas.py`.**

```
"""Canonical Manifest schema — v4.1."""
from datetime import datetime
from enum import Enum
from typing import Literal

from pydantic import BaseModel, ConfigDict, Field, field_validator

SchemaVersion = Literal["v4.1"]
ClipAxis = Literal["x", "y", "z"]
SceneGenerator = Literal["hunyuanworld", "stock"]


class JobStage(str, Enum):
    queued = "queued"
    orchestrating = "orchestrating"
    scene_gen = "scene_gen"
    objects_gen = "objects_gen"
    ready = "ready"
    failed = "failed"


class Color(BaseModel):
    color: str
```

```
    intensity: float = Field(ge=0.0, le=10.0)


class Fog(BaseModel):
    color: str
    near: float = Field(ge=0.0)
    far: float = Field(ge=0.0)


class Scene(BaseModel):
    generator: SceneGenerator
    scene_prompt: str
    stock_room_id: str | None = None
    scene_url: str
    ambient_light: Color
    fog: Fog | None = None


class Station(BaseModel):
    t: float = Field(ge=0.0, le=1.0)
    label: str
    pedestal_id: str | None
    preferred_yaw_deg: float


class Spline(BaseModel):
    control_points: list[list[float]] = Field(min_length=2)
    stations: list[Station] = Field(min_length=2)
    travel_time_s: float = Field(gt=0.0)

    @field_validator("control_points")
    @classmethod
    def _three_d(cls, v: list[list[float]]) -> list[list[float]]:
        for p in v:
            if len(p) != 3:
                raise ValueError(f"control point must be [x,y,z], got {p}")
        return v


class PedestalSpec(BaseModel):
    id: str
    position: list[float]
    height: float = Field(gt=0.0)
    object_id: str

    @field_validator("position")
    @classmethod
    def _three_d(cls, v: list[float]) -> list[float]:
        if len(v) != 3:
            raise ValueError(f"position must be [x,y,z], got {v}")
        return v
```

```python
class ObjectSpec(BaseModel):
    id: str
    label: str
    placard_description: str
    image_prompt: str
    image_url: str | None = None
    glb_url: str | None = None
    rotation_y_deg: float = 0.0
    scale: float = Field(gt=0.0, default=0.1)
    dissectable: bool = True
    preferred_clip_axis: ClipAxis = "y"
    preferred_clip_t: float = Field(ge=0.0, le=1.0, default=0.5)


class Timing(BaseModel):
    orchestrator_ms: int = 0
    scene_gen_ms: int = 0
    objects_gen_ms: dict[str, int] = Field(default_factory=dict)
    total_ms: int = 0


class StageError(BaseModel):
    stage: str
    object_id: str | None = None
    detail: str


class Manifest(BaseModel):
    model_config = ConfigDict(extra="forbid")

    schema_version: SchemaVersion = "v4.1"
    exhibit_id: str
    created_at: datetime
    user_prompt: str
    title: str
    description: str
    scene: Scene
    spline: Spline
    pedestals: list[PedestalSpec]
    objects: list[ObjectSpec]
    stage: JobStage
    timing: Timing = Field(default_factory=Timing)
    errors: list[StageError] = Field(default_factory=list)
```

- [ ] **Step 4: Run tests, verify pass.**

```
cd /mnt/3D_Mapping/backend && uv run pytest tests/test_schemas.py -v
```

Expected: 5 passed.

- [ ] **Step 5: Commit.**

```
cd /mnt/3D_Mapping
git add backend/app/schemas.py backend/tests/
git commit -m "feat(schemas): Manifest contract v4.1 with Pydantic validation"
```

Task 4: Exhibits API stubs (501)

Files: `backend/app/api/exhibits.py`, modify `backend/app/main.py`, `backend/tests/test_exhibits_api.py`.

- [ ] **Step 1: Write failing tests.**

```
"""Exhibits API tests — M0 expects 501 from every ML stage."""
from fastapi.testclient import TestClient

from app.main import app

client = TestClient(app)


def test_create_returns_501_in_m0() -> None:
    r = client.post("/api/exhibits", json={"prompt": "a Victorian study"})
    assert r.status_code == 501
    assert r.json()["detail"]["stage"] == "orchestrator"


def test_rejects_empty_prompt() -> None:
    r = client.post("/api/exhibits", json={"prompt": ""})
    assert r.status_code == 422


def test_rejects_missing_prompt() -> None:
    r = client.post("/api/exhibits", json={})
    assert r.status_code == 422


def test_unknown_exhibit_404() -> None:
    r = client.get("/api/exhibits/01HZ8X9A2B0000000000000001")
    assert r.status_code == 404
```

- [ ] **Step 2: Verify fail.** `uv run pytest tests/test_exhibits_api.py -v` → import error or all fail.
- [ ] **Step 3: Implement `backend/app/api/exhibits.py`.**

```
"""M0 stub — every ML stage returns 501."""
from fastapi import APIRouter, HTTPException
```

```python
from pydantic import BaseModel, Field
from sse_starlette.sse import EventSourceResponse

router = APIRouter(prefix="/api", tags=["exhibits"])


class CreateExhibitRequest(BaseModel):
    prompt: str = Field(min_length=1, max_length=500)


@router.post("/exhibits", status_code=501)
def create_exhibit(req: CreateExhibitRequest) -> None:
    raise HTTPException(
        status_code=501,
        detail={"stage": "orchestrator", "detail": "Not yet implemented (M0)"},
    )


@router.get("/exhibits/{exhibit_id}")
def get_exhibit(exhibit_id: str) -> None:
    raise HTTPException(status_code=404, detail={"exhibit_id": exhibit_id, "detail":
"not found"})


@router.get("/exhibits/{exhibit_id}/events")
async def exhibit_events(exhibit_id: str) -> EventSourceResponse:
    async def gen():
        yield {"event": "not_implemented", "data": exhibit_id}
    return EventSourceResponse(gen())
```

- [ ] **Step 4: Register in `backend/app/main.py`.**

```python
from fastapi import FastAPI

from .api.exhibits import router as exhibits_router
from .api.health import router as health_router

app = FastAPI(title="museums-backend", version="0.1.0")
app.include_router(health_router)
app.include_router(exhibits_router)


@app.get("/")
def root() -> dict[str, str]:
    return {"name": "museums-backend", "docs": "/docs"}
```

- [ ] **Step 5: Run tests, verify pass.** `uv run pytest tests/ -v` → all pass.
- [ ] **Step 6: Commit.**

```
cd /mnt/3D_Mapping
git add backend/
git commit -m "feat(api): exhibits routes returning 501 (M0 wiring)"
```

Task 5: Frontend scaffold (Vite + React 19 + TS)

Files: entire `frontend/` directory via Vite template.

- [ ] **Step 1: Initialize.**

```
cd /mnt/3D_Mapping
pnpm create vite frontend --template react-ts --no-git
cd frontend
pnpm install
```

- [ ] **Step 2: Add deps.**

```
cd /mnt/3D_Mapping/frontend
pnpm add three @types/three zustand react-router-dom
pnpm add -D tailwindcss@^3 postcss autoprefixer vitest @vitest/ui jsdom
pnpx tailwindcss init -p
```

- [ ] **Step 3: Replace `frontend/tailwind.config.js`.**

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ["./index.html", "./src/**/*.{js,ts,jsx,tsx}"],
  theme: { extend: {} },
  plugins: [],
};
```

- [ ] **Step 4: Replace `frontend/src/index.css`.**

```
@tailwind base;
@tailwind components;
@tailwind utilities;
html, body, #root { height: 100%; margin: 0; }
body { background: #0a0a0a; color: #e5e5e5; font-family: system-ui, sans-serif; }
```

- [ ] **Step 5: Replace `frontend/src/App.tsx`.**

```
import { BrowserRouter, Route, Routes } from "react-router-dom";
import Landing from "./pages/Landing";
import Generate from "./pages/Generate";
import Exhibit from "./pages/Exhibit";
```

```
export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Landing />} />
        <Route path="/generate" element={<Generate />} />
        <Route path="/exhibit/:id" element={<Exhibit />} />
      </Routes>
    </BrowserRouter>
  );
}
```

- [ ] **Step 6: Three page stubs.**

`frontend/src/pages/Landing.tsx`:

```
export default function Landing() {
  return (
    <main className="p-8">
      <h1 className="text-4xl font-bold">Interactive AI Museums</h1>
      <p className="mt-4">Type a prompt. Step inside. Take things apart.</p>
      <a href="/generate" className="mt-8 inline-block underline">Try it →</a>
    </main>
  );
}
```

`frontend/src/pages/Generate.tsx`:

```
export default function Generate() {
  return (
    <main className="p-8">
      <h1 className="text-2xl">Generate an exhibit</h1>
      <p className="mt-4 text-yellow-400">501 — orchestrator not yet implemented
(M0)</p>
    </main>
  );
}
```

`frontend/src/pages/Exhibit.tsx`:

```
import { useParams } from "react-router-dom";

export default function Exhibit() {
  const { id } = useParams();
  return (
    <main className="p-8">
      <h1 className="text-2xl">Exhibit {id}</h1>
      <p className="mt-4 text-yellow-400">Viewer scaffolded in Phase 3.</p>
    </main>
```

```
  );
}
```

- [ ] **Step 7: Verify boot.**

```
cd /mnt/3D_Mapping/frontend && pnpm dev &
sleep 5
curl -fsS http://localhost:5173/ | head -5
pkill -f 'vite'
```

Expected: HTML response containing `<div id="root">`.

- [ ] **Step 8: Commit.**

```
cd /mnt/3D_Mapping
git add frontend/
git commit -m "feat(frontend): Vite + React 19 + TS scaffold with three pages"
```

Task 6: Typed API client + manifest types

Files: `frontend/src/api/types.ts`, `frontend/src/api/client.ts`.

- [ ] **Step 1: Write `frontend/src/api/types.ts`.** Mirrors backend `schemas.py`.

```
export type JobStage = "queued" | "orchestrating" | "scene_gen" | "objects_gen" |
"ready" | "failed";
export type ClipAxis = "x" | "y" | "z";
export type SceneGenerator = "hunyuanworld" | "stock";

export interface Color { color: string; intensity: number; }
export interface Fog { color: string; near: number; far: number; }

export interface Scene {
  generator: SceneGenerator;
  scene_prompt: string;
  stock_room_id: string | null;
  scene_url: string;
  ambient_light: Color;
  fog: Fog | null;
}

export interface Station {
  t: number; label: string; pedestal_id: string | null; preferred_yaw_deg: number;
}

export interface Spline {
  control_points: number[][];
  stations: Station[];
  travel_time_s: number;
```

```
}

export interface PedestalSpec {
  id: string; position: number[]; height: number; object_id: string;
}

export interface ObjectSpec {
  id: string;
  label: string;
  placard_description: string;
  image_prompt: string;
  image_url: string | null;
  glb_url: string | null;
  rotation_y_deg: number;
  scale: number;
  dissectable: boolean;
  preferred_clip_axis: ClipAxis;
  preferred_clip_t: number;
}

export interface Timing {
  orchestrator_ms: number;
  scene_gen_ms: number;
  objects_gen_ms: Record<string, number>;
  total_ms: number;
}

export interface StageError {
  stage: string; object_id: string | null; detail: string;
}

export interface Manifest {
  schema_version: "v4.1";
  exhibit_id: string;
  created_at: string;
  user_prompt: string;
  title: string;
  description: string;
  scene: Scene;
  spline: Spline;
  pedestals: PedestalSpec[];
  objects: ObjectSpec[];
  stage: JobStage;
  timing: Timing;
  errors: StageError[];
}
```

- [ ] **Step 2: Write `frontend/src/api/client.ts`.**

```
import type { Manifest } from "./types";
```

```
const BASE = import.meta.env.VITE_API_BASE ?? "http://localhost:8000";

export class ApiError extends Error {
  constructor(public status: number, public body: unknown) { super(`API ${status}`);
}
}

async function request<T>(path: string, init?: RequestInit): Promise<T> {
  const r = await fetch(BASE + path, {
    ...init,
    headers: { "content-type": "application/json", ...(init?.headers ?? {}) },
  });
  if (!r.ok) {
    const body = await r.json().catch(() => null);
    throw new ApiError(r.status, body);
  }
  return r.json();
}

export async function createExhibit(prompt: string) {
  return request<{ exhibit_id: string; events_url: string }>("/api/exhibits", {
    method: "POST", body: JSON.stringify({ prompt }),
  });
}

export async function getExhibit(id: string): Promise<Manifest> {
  return request(`/api/exhibits/${id}`);
}
```

- [ ] **Step 3: Commit.**

```
cd /mnt/3D_Mapping
git add frontend/src/api/
git commit -m "feat(frontend): typed API client + Manifest types"
```

Task 7: Reference repos (clone)

Files: `references/TRELLIS.2/`, `references/HunyuanWorld/`, `references/supersplat/`, `references/splatviz/`, `references/README.md`.

- [ ] **Step 1: Shallow-clone.**

```
cd /mnt/3D_Mapping/references
git clone --depth=1 https://github.com/microsoft/TRELLIS.2.git
git clone --depth=1 https://github.com/Tencent-Hunyuan/HunyuanWorld-1.0.git
HunyuanWorld
git clone --depth=1 https://github.com/playcanvas/supersplat.git
git clone --depth=1 https://github.com/Florian-Barthel/splatviz.git
```

- [ ] **Step 2: Write `references/README.md`** (force-added since `references/` is gitignored).

```
# References (gitignored)

Local clones for study. Not bundled at build time; not imported as deps.

| Repo | Source | License | Use |
|---|---|---|---|
| TRELLIS.2 | microsoft/TRELLIS.2 | MIT | Object generator (Spike A + production) |
| HunyuanWorld | Tencent-Hunyuan/HunyuanWorld-1.0 | MIT-style | Scene generator
(Spike B + production if pass) |
| supersplat | playcanvas/supersplat | MIT | Splat viewer (study only) |
| splatviz | Florian-Barthel/splatviz | MIT | Python splat viewer (study only) |

Refresh: `cd references && rm -rf <repo> && git clone --depth=1 <url>`.
```

- [ ] **Step 3: Verify clones.**

```
ls /mnt/3D_Mapping/references/TRELLIS.2/README.md
ls /mnt/3D_Mapping/references/HunyuanWorld/README.md
```

- [ ] **Step 4: Commit README only.**

```
cd /mnt/3D_Mapping
git add -f references/README.md
git commit -m "docs(refs): document reference repo layout"
```

Task 8: Modal sanity (auth + H100 access)

Files: `modal/shared.py`, `modal/sanity.py`.

- [ ] **Step 1: Install Modal CLI + auth.**

```
pip install modal
modal token new
```

Token stored in `~/.modal.toml`.

- [ ] **Step 2: Write `modal/shared.py`.**

```
"""Shared Modal app + image defs."""
import modal

app = modal.App("museums-ml")

base_image = (
    modal.Image.debian_slim(python_version="3.12")
```

```
    .apt_install("git", "ffmpeg", "libgl1")
    .pip_install("torch==2.4.0", "torchvision==0.19.0", "numpy", "pillow",
"trimesh")
)

volume = modal.Volume.from_name("museums-exhibits", create_if_missing=True)
DATA_PATH = "/data"
```

- [ ] **Step 3: Write `modal/sanity.py`.**

```
"""GPU sanity check."""
import modal

from .shared import app, base_image, volume, DATA_PATH


@app.function(image=base_image, gpu="H100", volumes={DATA_PATH: volume},
timeout=120)
def gpu_sanity() -> dict:
    import subprocess
    out = subprocess.check_output(
        ["nvidia-smi", "--query-gpu=name,memory.total", "--format=csv,noheader"]
    )
    return {"gpu": out.decode().strip()}


@app.local_entrypoint()
def main() -> None:
    print(gpu_sanity.remote())
```

- [ ] **Step 4: Run.**

```
cd /mnt/3D_Mapping && modal run modal/sanity.py
```

Expected: `{'gpu': 'NVIDIA H100 80GB HBM3, 81559 MiB'}` (or similar).

- [ ] **Step 5: Commit.**

```
cd /mnt/3D_Mapping
git add modal/shared.py modal/sanity.py
git commit -m "feat(modal): sanity function — H100 access confirmed"
```

Task 9: Spike A — TRELLIS.2 interiors (Day 1)

Goal: Validate TRELLIS.2's "internal structures" claim across 10 prompts.

Files: `modal/spike_a_trellis.py`, `spikes/spike_a/` (gitignored), `SPIKE_RESULT.md`.

- [ ] **Step 1: Read TRELLIS.2 docs.**

```
cat /mnt/3D_Mapping/references/TRELLIS.2/README.md | head -200
ls /mnt/3D_Mapping/references/TRELLIS.2/example*.py
cat /mnt/3D_Mapping/references/TRELLIS.2/example.py
```

Note: exact Python API. Look for `from trellis...import`, the pipeline call signature, and the export method.

- [ ] **Step 2: Write `modal/spike_a_trellis.py`** adapted to actual TRELLIS.2 API found in step 1.

```
"""Spike A — TRELLIS.2 interior validation."""
from pathlib import Path

import modal

from .shared import app, volume, DATA_PATH

SPIKE_PROMPTS = [
    "a pomegranate cross-sectioned, seeds visible, photographic quality, plain white
background, single subject, no text",
    "a vintage mechanical pocket watch, back cover removed, gears and balance wheel
exposed, plain white background, single subject, no text",
    "a human heart, anatomical, chambers visible, hero shot, plain white background,
single subject, no text",
    "an apple sliced in half, core visible, plain white background, single subject,
no text",
    "a Gothic cathedral interior nave, model, plain white background, single
subject, no text",
    "a Russian nesting doll, opened, all layers visible, plain white background,
single subject, no text",
    "an avocado halved, pit centered, plain white background, single subject, no
text",
    "a desktop PC tower with side panel removed, motherboard and GPU visible, plain
white background, single subject, no text",
    "a Roman amphitheater, ruined, arcade level visible, model, plain white
background, single subject, no text",
    "a beehive cross-sectioned, honeycomb structure visible, plain white background,
single subject, no text",
]

trellis_image = (
    modal.Image.debian_slim(python_version="3.12")
    .apt_install("git", "ffmpeg", "libgl1", "wget")
    .pip_install(
        "torch==2.4.0", "torchvision==0.19.0", "numpy", "pillow", "trimesh",
        "diffusers>=0.30", "transformers>=4.44", "accelerate", "safetensors",
        "huggingface_hub", "imageio", "imageio-ffmpeg", "scipy", "tqdm",
    )
    .run_commands(
```

```
        "cd /opt && git clone --depth=1 https://github.com/microsoft/TRELLIS.2.git",
        "cd /opt/TRELLIS.2 && pip install -e .",
    )
)


@app.function(image=trellis_image, gpu="H100", volumes={DATA_PATH: volume},
timeout=1200)
def spike_a(prompts: list[str]) -> dict:
    import time
    import torch
    from diffusers import FluxPipeline
    from PIL import Image as PILImage

    out_dir = Path(DATA_PATH) / "spike_a"
    out_dir.mkdir(parents=True, exist_ok=True)

    flux = FluxPipeline.from_pretrained(
        "black-forest-labs/FLUX.1-schnell", torch_dtype=torch.bfloat16
    ).to("cuda")

    image_paths = []
    for i, p in enumerate(prompts):
        t0 = time.time()
        img = flux(p, num_inference_steps=4, guidance_scale=0.0, height=1024,
width=1024).images[0]
        img_path = out_dir / f"prompt_{i:02d}_hero.png"
        img.save(img_path)
        image_paths.append(img_path)
        print(f"[FLUX {i}] {time.time()-t0:.1f}s")

    del flux
    torch.cuda.empty_cache()

    # Adapt below to actual TRELLIS.2 API confirmed in step 1.
    from trellis.pipelines import TrellisImageTo3DPipeline  # CONFIRM
    trellis = TrellisImageTo3DPipeline.from_pretrained("microsoft/TRELLIS-image-
large").to("cuda")

    results = []
    for i, img_path in enumerate(image_paths):
        t0 = time.time()
        img = PILImage.open(img_path)
        out = trellis.run(img)  # CONFIRM call signature
        glb_path = out_dir / f"prompt_{i:02d}_object.glb"
        # CONFIRM export call: e.g., out['mesh'][0].export(glb_path) or
trellis.export_glb(...)
        out["mesh"][0].export(str(glb_path))
        dt = time.time() - t0
        results.append({"prompt_idx": i, "glb_path": str(glb_path), "elapsed_s":
dt})
        print(f"[TRELLIS {i}] {dt:.1f}s")
```

```
    volume.commit()
    return {"results": results}


@app.local_entrypoint()
def main() -> None:
    import json
    print(json.dumps(spike_a.remote(SPIKE_PROMPTS), indent=2))
```

- [ ] **Step 3: Run spike.**

```
cd /mnt/3D_Mapping
modal run modal/spike_a_trellis.py
```

Expected wall-clock: 10–15 min for 10 prompts.

- [ ] **Step 4: Download outputs.**

```
mkdir -p /mnt/3D_Mapping/spikes/spike_a
modal volume get museums-exhibits spike_a /mnt/3D_Mapping/spikes/spike_a/
ls /mnt/3D_Mapping/spikes/spike_a/
```

Expected: 10 PNGs + 10 GLBs.

- [ ] **Step 5: Manually score each GLB in Blender.**

For each `prompt_NN_object.glb` : Import → wireframe view → slice with Boolean → score 0/1/2 (none / some / meaningful interior).

- [ ] **Step 6: Tally + decide.**

- Pass: sum ≥ 12, twos ≥ 4 → green-light dissection.
- 8–11: scope-trim (food + anatomy only).
- < 8: pivot (dissection thesis dies).

- [ ] **Step 7: Write Spike A section in `SPIKE_RESULT.md` .**

```
# Spike Results — Week 1

## Spike A: TRELLIS.2 interiors

| # | Prompt | Score | Notes |
|---|---|---|---|
| 0 | pomegranate | _ | _ |
| 1 | pocket watch | _ | _ |
| ... | ... | ... | ... |

**Total:** _ / 20
```

```
**Twos:** _ / 10
**Result:** PASS / SCOPE-TRIM / PIVOT
**Decision:** _
```

- [ ] **Step 8: Commit.**

```
cd /mnt/3D_Mapping
git add modal/spike_a_trellis.py SPIKE_RESULT.md
git commit -m "spike(a): TRELLIS.2 interior validation on 10 prompts"
```

Task 10: Spike B — HunyuanWorld walkability (Day 2)

Goal: Validate that HunyuanWorld scenes look coherent under camera motion along a spline.

Files: `modal/spike_b_hunyuan.py`, `spikes/spike_b/`, `SPIKE_RESULT.md`.

- [ ] **Step 1: Read HunyuanWorld API.**

```
cat /mnt/3D_Mapping/references/HunyuanWorld/README.md
ls /mnt/3D_Mapping/references/HunyuanWorld/*.py
```

Note input format, output format (likely GLB or PLY), pipeline class name.

- [ ] **Step 2: Write `modal/spike_b_hunyuan.py`** adapted to step 1.

```
"""Spike B — HunyuanWorld walkability test."""
from pathlib import Path

import modal

from .shared import app, volume, DATA_PATH

SCENE_PROMPTS = [
    "Victorian study interior, oak panelling, brass fixtures, warm afternoon light
through a tall window, no people, no foreground objects, empty pedestals at center-
room, photo-real",
    "apothecary's workshop, dimly lit shelves with bottles in the background only,
no people, no foreground objects, empty pedestals at center-room, photo-real",
    "biology classroom, fluorescent lighting, empty desks, blackboard, no people, no
foreground objects, empty pedestals at center-room, photo-real",
]

hunyuan_image = (
    modal.Image.debian_slim(python_version="3.12")
    .apt_install("git", "ffmpeg", "libgl1", "wget")
    .pip_install("torch==2.4.0", "torchvision==0.19.0", "numpy", "pillow",
"trimesh")
    .run_commands(
        "cd /opt && git clone --depth=1 https://github.com/Tencent-
```

```
Hunyuan/HunyuanWorld-1.0.git HunyuanWorld",
        "cd /opt/HunyuanWorld && pip install -r requirements.txt && pip install -e
.",
    )
)


@app.function(image=hunyuan_image, gpu="H100", volumes={DATA_PATH: volume},
timeout=2400)
def spike_b(prompts: list[str]) -> dict:
    import time
    from hunyuanworld.pipeline import HunyuanWorldPipeline  # CONFIRM

    pipe = HunyuanWorldPipeline.from_pretrained("Tencent-Hunyuan/HunyuanWorld-
1.0").to("cuda")

    out_dir = Path(DATA_PATH) / "spike_b"
    out_dir.mkdir(parents=True, exist_ok=True)

    results = []
    for i, p in enumerate(prompts):
        t0 = time.time()
        scene = pipe.generate(p, seed=42)  # CONFIRM
        glb_path = out_dir / f"scene_{i:02d}.glb"
        scene.export(str(glb_path))  # CONFIRM
        dt = time.time() - t0
        results.append({"prompt_idx": i, "glb_path": str(glb_path), "elapsed_s":
dt})
        print(f"[{i}] {dt:.1f}s")

    volume.commit()
    return {"results": results}


@app.local_entrypoint()
def main() -> None:
    import json
    print(json.dumps(spike_b.remote(SCENE_PROMPTS), indent=2))
```

- [ ] **Step 3: Run.**

```
cd /mnt/3D_Mapping
modal run modal/spike_b_hunyuan.py
```

Expected: ~8 min for 3 scenes.

- [ ] **Step 4: Download.**

```
mkdir -p /mnt/3D_Mapping/spikes/spike_b
modal volume get museums-exhibits spike_b /mnt/3D_Mapping/spikes/spike_b/
```

- [ ] **Step 5: Render fly-throughs.**

For each `scene_NN.glb`:

1. Open in Blender.
2. Add Bezier curve with ~4 control points within the room.
3. Camera with Follow Path constraint, 180 frames at 30 fps.
4. Render → MP4 to `spikes/spike_b/flythrough_NN.mp4`.

(Or write `spikes/spike_b/flythrough.py` Blender script.)

- [ ] **Step 6: Score each fly-through.** Three criteria:

1. Generated without errors in < 8 min.
2. No tearing / holes / aliased garbage when camera moves.
3. Pedestals can be plausibly placed on a floor.

Pass = all three for at least 2 of 3 scenes.

- [ ] **Step 7: Tally + decide.**
- 3/3 pass → green-light HunyuanWorld primary.
- 1–2 pass → one prompt-engineering retry; if still < 3, drop to Approach C.
- 0 pass → drop to Approach C.
- [ ] **Step 8: Write Spike B section in `SPIKE_RESULT.md`.**
- [ ] **Step 9: Commit.**

```
cd /mnt/3D_Mapping
git add modal/spike_b_hunyuan.py SPIKE_RESULT.md
git commit -m "spike(b): HunyuanWorld walkability scored 3 scenes"
```

Task 11: Spike C — Three.js capped clipping (Day 3)

Goal: Validate the dissection moat at the rendering layer.

Files: `spikes/spike_c/index.html`, `spikes/spike_c/src/main.ts`, `spikes/spike_c/package.json`.

- [ ] **Step 1: Scaffold.**

```
cd /mnt/3D_Mapping/spikes
pnpm create vite spike_c --template vanilla-ts --no-git
cd spike_c
pnpm install
pnpm add three @types/three lil-gui
mkdir -p public
```

- [ ] **Step 2: Copy best Spike A GLB** to `spike_c/public/object.glb`.

```
cp /mnt/3D_Mapping/spikes/spike_a/prompt_<best>_object.glb
/mnt/3D_Mapping/spikes/spike_c/public/object.glb
```

- [ ] **Step 3: Write `spikes/spike_c/src/main.ts`.**

```
import * as THREE from "three";
import { GLTFLoader } from "three/addons/loaders/GLTFLoader.js";
import { OrbitControls } from "three/addons/controls/OrbitControls.js";
import { RGBELoader } from "three/addons/loaders/RGBELoader.js";
import GUI from "lil-gui";

const canvas = document.querySelector<HTMLCanvasElement>("#c")!;
const renderer = new THREE.WebGLRenderer({ canvas, antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.setPixelRatio(window.devicePixelRatio);
renderer.localClippingEnabled = true;

const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(45, window.innerWidth /
window.innerHeight, 0.01, 100);
camera.position.set(0.5, 0.5, 0.8);

new OrbitControls(camera, renderer.domElement);
new RGBELoader().load(
  "https://threejs.org/examples/textures/equirectangular/royal_esplanade_1k.hdr",
  (hdr) => { hdr.mapping = THREE.EquirectangularReflectionMapping; scene.environment
= hdr; },
);

const clipPlane = new THREE.Plane(new THREE.Vector3(0, -1, 0), 0);

const capMat = new THREE.MeshBasicMaterial({
  color: 0xaa6633,
  stencilWrite: true, stencilRef: 1, stencilFunc: THREE.EqualStencilFunc,
  stencilFail: THREE.KeepStencilOp, stencilZFail: THREE.KeepStencilOp, stencilZPass:
THREE.KeepStencilOp,
});
const capMesh = new THREE.Mesh(new THREE.PlaneGeometry(10, 10), capMat);
capMesh.rotation.x = -Math.PI / 2;
capMesh.renderOrder = 1;
scene.add(capMesh);

new GLTFLoader().load("/object.glb", (gltf) => {
  gltf.scene.traverse((o) => {
    const mesh = o as THREE.Mesh;
    if (mesh.isMesh) {
      const m = mesh.material as THREE.MeshStandardMaterial;
      m.clippingPlanes = [clipPlane];
      m.stencilWrite = true;
      m.stencilRef = 1;
```

```
      m.stencilFunc = THREE.AlwaysStencilFunc;
      m.stencilZPass = THREE.ReplaceStencilOp;
      m.side = THREE.DoubleSide;
    }
  });
  scene.add(gltf.scene);
});

const gui = new GUI();
const params = { clipT: 0.5 };
gui.add(params, "clipT", 0, 1, 0.01).onChange((v: number) => {
  clipPlane.constant = (v - 0.5) * 1.0;
  capMesh.position.y = clipPlane.constant;
});

(function loop() {
  requestAnimationFrame(loop);
  renderer.render(scene, camera);
})();
```

- [ ] **Step 4: Update `spikes/spike_c/index.html`.**

```
<!DOCTYPE html>
<html><head><meta charset="utf-8"><title>Spike C</title>
<style>body{margin:0;background:#111}</style></head>
<body><canvas id="c"></canvas><script type="module" src="/src/main.ts"></script>
</body></html>
```

- [ ] **Step 5: Run.**

```
cd /mnt/3D_Mapping/spikes/spike_c && pnpm dev
```

Open browser → drag clipT slider → verify:

1. Object cross-sections smoothly.
2. Cap is filled (brown color) at the cut.
3. No flickering / Z-fighting.
4. 60 fps in browser dev tools.

- [ ] **Step 6: Score + write Spike C section in `SPIKE_RESULT.md`.**

```
## Spike C: Three.js capped clipping

- Clipping: PASS / FAIL
- Capping: PASS / FAIL
- 60fps: PASS / FAIL
- Result: PASS / DEGRADE (no caps) / PIVOT (no dissection)
```

- [ ] **Step 7: Record short video** to `spikes/spike_c/recording.mp4`.

- [ ] **Step 8: Commit.**

```
cd /mnt/3D_Mapping
git add spikes/spike_c/src/ spikes/spike_c/index.html spikes/spike_c/package.json
SPIKE_RESULT.md
git commit -m "spike(c): three.js capped clipping proof-of-concept"
```

Task 12: Spike decision + handoff

- [ ] **Step 1: Read all three spike sections in `SPIKE_RESULT.md`.**
- [ ] **Step 2: Append decision section.**

```
## Final decision

- Spike A (TRELLIS.2): PASS / SCOPE-TRIM / FAIL — _
- Spike B (HunyuanWorld): PASS / SCOPE-TRIM / FAIL — _
- Spike C (capped clipping): PASS / DEGRADE / FAIL — _

**Build decision:** GREEN-LIGHT / SCOPE-TRIM / PIVOT
**Reasoning:** _
**Adjustments to spec/plan:** _

Next: Week 2 begins. Implement Tasks 13–17.
```

- [ ] **Step 3: Commit.**

```
cd /mnt/3D_Mapping
git add SPIKE_RESULT.md
git commit -m "spike: week-1 decision committed"
```

- [ ] **Step 4: Branch.**
- GREEN-LIGHT → Phase 2.
- SCOPE-TRIM → update spec + plan inline; commit; proceed.
- PIVOT → halt plan; re-brainstorm.

Phase 2 — Week 2: Orchestrator + ML plumbing (M1, M2)

By end of week 2: real prompt → real `ready` manifest in ~10 min wall clock.

Task 13: Claude orchestrator

Files: `backend/app/orchestrator/__init__.py`, `backend/app/orchestrator/prompts.py`, `backend/app/orchestrator/claude.py`, `backend/app/orchestrator/layout_templates.py`, `backend/tests/fixtures/orchestrator/few_shot/*.json`, `backend/tests/test_orchestrator.py`.

- [ ] **Step 1: Write `backend/app/orchestrator/__init__.py`** (empty).
- [ ] **Step 2: Write `backend/app/orchestrator/layout_templates.py`.**

```
"""Layout templates the orchestrator picks from."""
from typing import Literal

LayoutName = Literal["linear", "hub_and_spokes", "semicircle"]


def layout_description(name: LayoutName, n_objects: int) -> str:
    if name == "linear":
        return f"Linear gallery: {n_objects} pedestals on a line at z=-2, 1.5m
apart, centered on x. Spline starts at (0,1.7,4)."
    if name == "hub_and_spokes":
        return f"Hub-and-spokes: {n_objects} pedestals at radius 2.5m, evenly
angled. Spline visits each via center."
    if name == "semicircle":
        return f"Semicircle: {n_objects} pedestals on far half of 2.5m circle,
facing viewer. Spline at radius 1m walks past."
    raise ValueError(name)
```

- [ ] **Step 3: Create three few-shot fixtures.**

`backend/tests/fixtures/orchestrator/few_shot/anatomy.json` — complete valid Manifest for "An anatomy exhibit" with 3 objects (heart, brain, skeleton), hub-and-spokes layout. Use the schema literally.

Likewise `horology.json` (3 pocket watches, linear) and `botany.json` (pomegranate, apple, avocado, semicircle).

(Generate these from the `_valid_payload()` in test_schemas.py as a template; substitute content per exhibit. Each file ~2KB.)

- [ ] **Step 4: Write `backend/app/orchestrator/prompts.py`.**

```
"""Versioned orchestrator system prompt + few-shot loader."""
import json
from pathlib import Path

SYSTEM_PROMPT_V1 = """\
You are a scene designer for an interactive 3D museum demo. Given a user's one-line
prompt for an exhibit, produce a JSON manifest that fully specifies the experience.
```

```
The output JSON must conform exactly to the Manifest schema v4.1.

Your six jobs, in order:

1. REFUSAL CHECK. If the prompt requests NSFW content, real-person likeness, gore
   with photorealism, or hate content, return ONLY:
   {"refused": true, "reason": "<one short sentence>"}

2. PICK 3-5 OBJECTS. Each object must be a discrete, isolatable, hero-shot-able
   subject. Bias toward objects with plausible interior structure (mechanical,
   anatomical, layered, food with cores).

3. WRITE scene_prompt for HunyuanWorld. Describe AMBIENT features only. Explicitly
   EXCLUDE foreground objects. End with: "no people, no foreground objects,
   empty pedestals at center-room"

4. WRITE per-object image_prompt for FLUX.1-schnell. Use this scaffold exactly:
   "<thing>, hero shot, plain white background, centered, photographic quality,
    single subject, no text"
   Add dissection hint if relevant: "cross-sectioned", "back cover removed".

5. LAY OUT SPLINE + PEDESTALS. Pick a layout (linear / hub_and_spokes / semicircle)
   matching object count. Eye height 1.7m. Stations spaced for 5-8s travel.
   Coordinates: 8m x 8m room, origin at center, floor at y=0.

6. WRITE title (<=4 words) + description (1-2 sentences) for the share card.

Constants:
- schema_version = "v4.1"
- stage = "orchestrating"
- timing = all zeros (backend fills)
- errors = []
- scene.generator = "hunyuanworld" unless force_generator="stock"
- objects[*].image_url, glb_url = null

Output strict JSON. No prose. No backticks.

Few-shot examples follow.
"""


def load_few_shot() -> list[dict]:
    here = Path(__file__).resolve().parent.parent.parent
    d = here / "tests" / "fixtures" / "orchestrator" / "few_shot"
    return [json.loads(p.read_text()) for p in sorted(d.glob("*.json"))]


def build_user_message(prompt: str, force_generator: str | None = None) -> str:
    if force_generator == "stock":
        return (
            f"User prompt: {prompt!r}\n\nConstraint: scene.generator must be
\"stock\". "
```

```
            f"Pick scene.stock_room_id from: victorian_study_001, library_warm_001,
"
            f"workshop_industrial_001, gallery_white_001, classroom_neutral_001, "
            f"laboratory_clinical_001, kitchen_period_001, conservatory_botanic_001,
"
            f"attic_dim_001, pavilion_outdoor_001. Choose closest match."
        )
    return f"User prompt: {prompt!r}"
```

- [ ] **Step 5: Write `backend/app/orchestrator/claude.py`.**

```
"""Claude orchestrator with schema-retry."""
import json
from typing import Literal

from anthropic import Anthropic
from pydantic import ValidationError

from ..config import get_settings
from ..schemas import Manifest
from .prompts import SYSTEM_PROMPT_V1, build_user_message, load_few_shot

_client: Anthropic | None = None


def _client_lazy() -> Anthropic:
    global _client
    if _client is None:
        _client = Anthropic(api_key=get_settings().anthropic_api_key)
    return _client


def _build_messages(user_prompt: str, validation_error: str | None, force_generator:
str | None) -> list[dict]:
    examples = load_few_shot()
    fewshot_text = "\n\n".join(
        f"Example prompt: {ex['user_prompt']!r}\nExample manifest:\n{json.dumps(ex,
indent=2)}"
        for ex in examples
    )
    user_msg = build_user_message(user_prompt, force_generator)
    if validation_error is not None:
        user_msg += f"\n\nYour previous output failed Pydantic
validation:\n{validation_error}\n\nFix the issue and emit valid JSON."
    return [{"role": "user", "content": fewshot_text + "\n\n---\n\n" + user_msg}]


class OrchestratorRefused(Exception):
    def __init__(self, reason: str):
        super().__init__(reason)
        self.reason = reason
```

```
class OrchestratorFailed(Exception):
    def __init__(self, detail: str):
        super().__init__(detail)
        self.detail = detail


def orchestrate(
    user_prompt: str,
    exhibit_id: str,
    force_generator: Literal["hunyuanworld", "stock"] | None = None,
) -> Manifest:
    settings = get_settings()
    client = _client_lazy()

    last_error: str | None = None
    for _ in range(2):
        msgs = _build_messages(user_prompt, last_error, force_generator)
        resp = client.messages.create(
            model=settings.claude_model,
            max_tokens=4096,
            system=SYSTEM_PROMPT_V1,
            messages=msgs,
        )
        raw = resp.content[0].text.strip()
        if raw.startswith("```"):
            raw = raw.split("\n", 1)[1].rsplit("```", 1)[0]
        try:
            obj = json.loads(raw)
        except json.JSONDecodeError as e:
            last_error = f"Invalid JSON: {e}"
            continue
        if obj.get("refused"):
            raise OrchestratorRefused(obj.get("reason", "Refused without reason."))
        obj["exhibit_id"] = exhibit_id
        try:
            return Manifest.model_validate(obj)
        except ValidationError as e:
            last_error = str(e)
            continue
    raise OrchestratorFailed(last_error or "Two retries exhausted.")
```

- [ ] **Step 6: Write `backend/tests/test_orchestrator.py`.**

```
"""Orchestrator tests — few-shot validity + (optional) integration with Claude."""
import json
import os
from pathlib import Path

import pytest
```

```
from app.schemas import Manifest

FIXTURES = Path(__file__).parent / "fixtures" / "orchestrator" / "few_shot"


@pytest.mark.parametrize("name", ["anatomy", "horology", "botany"])
def test_few_shot_validates(name: str) -> None:
    payload = json.loads((FIXTURES / f"{name}.json").read_text())
    Manifest.model_validate(payload)


@pytest.mark.skipif(not os.environ.get("ANTHROPIC_API_KEY"), reason="needs API key")
def test_orchestrate_real_call() -> None:
    from app.orchestrator.claude import orchestrate
    m = orchestrate("A Victorian study with a pocket watch",
exhibit_id="TEST_INTEGRATION_001")
    assert m.exhibit_id == "TEST_INTEGRATION_001"
    assert 3 <= len(m.objects) <= 5
    assert m.scene.generator == "hunyuanworld"
```

- [ ] **Step 7: Run tests.**

```
cd /mnt/3D_Mapping/backend && uv run pytest tests/test_orchestrator.py -v
```

Expected: 3 few-shot tests pass.

- [ ] **Step 8: Run integration test (optional, costs ~$0.02).**

```
cd /mnt/3D_Mapping/backend
ANTHROPIC_API_KEY=sk-... uv run pytest
tests/test_orchestrator.py::test_orchestrate_real_call -v
```

- [ ] **Step 9: Commit.**

```
cd /mnt/3D_Mapping
git add backend/app/orchestrator/ backend/tests/fixtures/orchestrator/
backend/tests/test_orchestrator.py
git commit -m "feat(orchestrator): Claude planner + 3 few-shot exhibits"
```

Task 14: Modal scene_gen production

Files: `modal/scene_gen.py`.

- [ ] **Step 1: Write `modal/scene_gen.py`.** (Same template as Spike B, productionized.)

```
"""Production scene_gen — HunyuanWorld + stock loader."""
from pathlib import Path
```

```
import modal

from .shared import app, volume, DATA_PATH

hunyuan_image = (
    modal.Image.debian_slim(python_version="3.12")
    .apt_install("git", "ffmpeg", "libgl1", "wget")
    .pip_install("torch==2.4.0", "torchvision==0.19.0", "numpy", "pillow",
"trimesh")
    .run_commands(
        "cd /opt && git clone --depth=1 https://github.com/Tencent-
Hunyuan/HunyuanWorld-1.0.git HunyuanWorld",
        "cd /opt/HunyuanWorld && pip install -r requirements.txt && pip install -e
.",
    )
)


@app.function(image=hunyuan_image, gpu="H100", volumes={DATA_PATH: volume},
timeout=1200)
def scene_gen_hunyuan(scene_prompt: str, exhibit_id: str) -> dict:
    import time
    from hunyuanworld.pipeline import HunyuanWorldPipeline  # CONFIRM

    pipe = HunyuanWorldPipeline.from_pretrained("Tencent-Hunyuan/HunyuanWorld-
1.0").to("cuda")
    t0 = time.time()
    scene = pipe.generate(scene_prompt, seed=42)
    out_path = Path(DATA_PATH) / "exhibits" / exhibit_id / "scene" / "scene.glb"
    out_path.parent.mkdir(parents=True, exist_ok=True)
    scene.export(str(out_path))
    volume.commit()
    return {"scene_url": f"/exhibits/{exhibit_id}/scene/scene.glb", "elapsed_ms":
int((time.time()-t0)*1000)}


stock_image = base_image_with_stock_rooms = (
    modal.Image.debian_slim(python_version="3.12")
    .pip_install("trimesh", "pillow")
    .add_local_dir("/mnt/3D_Mapping/backend/stock_rooms/rooms",
remote_path="/opt/stock_rooms")
)


@app.function(image=stock_image, volumes={DATA_PATH: volume}, timeout=60)
def scene_gen_stock(stock_room_id: str, exhibit_id: str) -> dict:
    import shutil
    import time
    t0 = time.time()
    src = Path("/opt/stock_rooms") / f"{stock_room_id}.glb"
    if not src.exists():
```

```
        raise RuntimeError(f"Unknown stock room: {stock_room_id}")
    out_path = Path(DATA_PATH) / "exhibits" / exhibit_id / "scene" / "scene.glb"
    out_path.parent.mkdir(parents=True, exist_ok=True)
    shutil.copy(src, out_path)
    volume.commit()
    return {"scene_url": f"/exhibits/{exhibit_id}/scene/scene.glb", "elapsed_ms":
int((time.time()-t0)*1000)}
```

- [ ] **Step 2: Deploy + smoke test HunyuanWorld function.**

```
cd /mnt/3D_Mapping
modal deploy modal/scene_gen.py
modal run modal/scene_gen.py::scene_gen_hunyuan --scene-prompt "Victorian study,
empty, photo-real" --exhibit-id "TEST_SCENE_001"
```

Expected: `{"scene_url": "/exhibits/TEST_SCENE_001/scene/scene.glb", "elapsed_ms": ~240000}`.

- [ ] **Step 3: Skip stock test until Task 16 ships stock rooms.**
- [ ] **Step 4: Commit.**

```
cd /mnt/3D_Mapping
git add modal/scene_gen.py
git commit -m "feat(modal): scene_gen production (HunyuanWorld + stock loader)"
```

Task 15: Modal object_gen production

Files: `modal/object_gen.py`.

- [ ] **Step 1: Write `modal/object_gen.py`.**

```
"""Production object_gen — FLUX.1-schnell → TRELLIS.2."""
from pathlib import Path

import modal

from .shared import app, volume, DATA_PATH

object_image = (
    modal.Image.debian_slim(python_version="3.12")
    .apt_install("git", "ffmpeg", "libgl1", "wget")
    .pip_install(
        "torch==2.4.0", "torchvision==0.19.0", "numpy", "pillow", "trimesh",
        "diffusers>=0.30", "transformers>=4.44", "accelerate", "safetensors",
        "huggingface_hub", "imageio", "imageio-ffmpeg", "scipy", "tqdm",
    )
    .run_commands(
```

```
        "cd /opt && git clone --depth=1 https://github.com/microsoft/TRELLIS.2.git",
        "cd /opt/TRELLIS.2 && pip install -e .",
    )
)


@app.function(image=object_image, gpu="H100", volumes={DATA_PATH: volume},
timeout=300)
def object_gen(image_prompt: str, exhibit_id: str, object_id: str, resolution: int =
512) -> dict:
    import time
    import torch
    from diffusers import FluxPipeline
    from PIL import Image as PILImage

    out_dir = Path(DATA_PATH) / "exhibits" / exhibit_id / "objects"
    out_dir.mkdir(parents=True, exist_ok=True)
    t0 = time.time()

    flux = FluxPipeline.from_pretrained(
        "black-forest-labs/FLUX.1-schnell", torch_dtype=torch.bfloat16
    ).to("cuda")
    img = flux(image_prompt, num_inference_steps=4, guidance_scale=0.0, height=1024,
width=1024).images[0]
    img_path = out_dir / f"{object_id}.png"
    img.save(img_path)
    del flux
    torch.cuda.empty_cache()

    from trellis.pipelines import TrellisImageTo3DPipeline  # CONFIRM
    trellis = TrellisImageTo3DPipeline.from_pretrained("microsoft/TRELLIS-image-
large").to("cuda")
    out = trellis.run(img, resolution=resolution)
    glb_path = out_dir / f"{object_id}.glb"
    out["mesh"][0].export(str(glb_path))  # CONFIRM call

    volume.commit()
    return {
        "image_url": f"/exhibits/{exhibit_id}/objects/{object_id}.png",
        "glb_url": f"/exhibits/{exhibit_id}/objects/{object_id}.glb",
        "elapsed_ms": int((time.time() - t0) * 1000),
    }
```

- [ ] **Step 2: Deploy + smoke test.**

```
cd /mnt/3D_Mapping
modal deploy modal/object_gen.py
modal run modal/object_gen.py::object_gen --image-prompt "antique brass pocket
watch, hero shot, plain white background" --exhibit-id "TEST_OBJ_001" --object-id
"watch_1"
```

Expected: latency ~35s; GLB written to volume.

- [ ] **Step 3: Commit.**

```
cd /mnt/3D_Mapping
git add modal/object_gen.py
git commit -m "feat(modal): object_gen production (FLUX -> TRELLIS.2)"
```

Task 16: Stock-room library

Files: `backend/stock_rooms/manifest.json`, `backend/stock_rooms/rooms/*.glb`.

- [ ] **Step 1: Source 10 stock rooms.**

From Polyhaven (CC0) or Sketchfab CC0. Target IDs: `victorian_study_001, library_warm_001, workshop_industrial_001, gallery_white_001, classroom_neutral_001, laboratory_clinical_001, kitchen_period_001, conservatory_botanic_001, attic_dim_001, pavilion_outdoor_001`. 5–20 MB each. Save to `backend/stock_rooms/rooms/<id>.glb`.

- [ ] **Step 2: Write `backend/stock_rooms/manifest.json`.**

```
[
  {"id": "victorian_study_001", "themes": ["historical", "study", "warm-light"],
"style": "photoreal", "size_m": [6, 3, 6], "credit": "Polyhaven CC0"},
  {"id": "library_warm_001", "themes": ["historical", "library", "warm-light"],
"style": "photoreal", "size_m": [8, 4, 6], "credit": "Polyhaven CC0"},
  {"id": "workshop_industrial_001", "themes": ["industrial", "workshop", "neutral-
light"], "style": "photoreal", "size_m": [10, 4, 8], "credit": "Polyhaven CC0"},
  {"id": "gallery_white_001", "themes": ["modern", "gallery", "neutral-light"],
"style": "photoreal", "size_m": [8, 4, 8], "credit": "Polyhaven CC0"},
  {"id": "classroom_neutral_001", "themes": ["educational", "classroom"], "style":
"photoreal", "size_m": [8, 3, 8], "credit": "Polyhaven CC0"},
  {"id": "laboratory_clinical_001", "themes": ["clinical", "scientific"], "style":
"photoreal", "size_m": [8, 3, 6], "credit": "Polyhaven CC0"},
  {"id": "kitchen_period_001", "themes": ["historical", "domestic"], "style":
"photoreal", "size_m": [6, 3, 6], "credit": "Polyhaven CC0"},
  {"id": "conservatory_botanic_001", "themes": ["greenhouse", "botanic", "natural-
light"], "style": "photoreal", "size_m": [8, 5, 6], "credit": "Polyhaven CC0"},
  {"id": "attic_dim_001", "themes": ["historical", "atmospheric", "dim-light"],
"style": "photoreal", "size_m": [6, 3, 6], "credit": "Polyhaven CC0"},
  {"id": "pavilion_outdoor_001", "themes": ["outdoor", "natural-light"], "style":
"photoreal", "size_m": [10, 4, 10], "credit": "Polyhaven CC0"}
]
```

- [ ] **Step 3: Test stock fallback.**

```
cd /mnt/3D_Mapping
modal run modal/scene_gen.py::scene_gen_stock --stock-room-id "victorian_study_001"
--exhibit-id "TEST_STOCK_001"
```

- [ ] **Step 4: Commit (GLBs gitignored).**

```
cd /mnt/3D_Mapping
git add backend/stock_rooms/manifest.json
git commit -m "feat(stock): 10-room library metadata (GLBs gitignored)"
```

Task 17: Backend → ML orchestration pipeline

Files: `backend/app/modal_client/{__init__.py,_modal.py,scene.py,object.py}`, `backend/app/storage/{__init__.py,volume.py}`, `backend/app/routing.py`, modify `backend/app/api/exhibits.py`, modify `backend/tests/test_exhibits_api.py`.

- [ ] **Step 1: Write `backend/app/modal_client/__init__.py`** (empty).

- [ ] **Step 2: Write `backend/app/modal_client/_modal.py`.**

```
"""Modal app handle for backend → ML function dispatch."""
from ..config import get_settings

app_name = get_settings().modal_app_name
```

- [ ] **Step 3: Write `backend/app/modal_client/scene.py`.**

```
"""Scene-gen wrapper (called from backend)."""
import modal

from ._modal import app_name

_hunyuan = modal.Function.lookup(app_name, "scene_gen_hunyuan")
_stock = modal.Function.lookup(app_name, "scene_gen_stock")


def call_scene_hunyuan(scene_prompt: str, exhibit_id: str) -> dict:
    return _hunyuan.remote(scene_prompt, exhibit_id)


def call_scene_stock(stock_room_id: str, exhibit_id: str) -> dict:
    return _stock.remote(stock_room_id, exhibit_id)
```

- [ ] **Step 4: Write `backend/app/modal_client/object.py`.**

```
"""Object-gen wrapper."""
import modal

from ._modal import app_name

_object_gen = modal.Function.lookup(app_name, "object_gen")
```

```
def spawn_object(image_prompt: str, exhibit_id: str, object_id: str, resolution: int
= 512) -> "modal.FunctionCall":
    return _object_gen.spawn(image_prompt, exhibit_id, object_id, resolution)
```

- [ ] **Step 5: Write `backend/app/storage/__init__.py`** (empty), then `backend/app/storage/volume.py`.

```
"""Manifest I/O on the Modal volume."""
from pathlib import Path

from ..config import get_settings
from ..schemas import Manifest


def exhibit_dir(exhibit_id: str) -> Path:
    return Path(get_settings().data_dir) / "exhibits" / exhibit_id


def manifest_path(exhibit_id: str) -> Path:
    return exhibit_dir(exhibit_id) / "manifest.json"


def write_manifest(m: Manifest) -> None:
    p = manifest_path(m.exhibit_id)
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(m.model_dump_json(indent=2))


def read_manifest(exhibit_id: str) -> Manifest | None:
    p = manifest_path(exhibit_id)
    if not p.exists():
        return None
    return Manifest.model_validate_json(p.read_text())
```

- [ ] **Step 6: Write `backend/app/routing.py`.**

```
"""Full pipeline orchestration with degradation."""
import json
import time
from typing import AsyncIterator
from uuid import uuid4

from .modal_client.object import spawn_object
from .modal_client.scene import call_scene_hunyuan, call_scene_stock
from .orchestrator.claude import (
    OrchestratorFailed,
    OrchestratorRefused,
    orchestrate,
```

```
)
from .schemas import JobStage, Manifest, StageError
from .storage.volume import write_manifest


def _new_exhibit_id() -> str:
    return uuid4().hex


async def run_pipeline(user_prompt: str) -> AsyncIterator[dict]:
    """Yields SSE events. Final event has stage=ready or stage=failed."""
    exhibit_id = _new_exhibit_id()
    t_start = time.time()

    yield {"event": "stage", "data": json.dumps({"stage": "orchestrating",
"exhibit_id": exhibit_id})}

    try:
        manifest = orchestrate(user_prompt, exhibit_id=exhibit_id)
    except OrchestratorRefused as e:
        yield {"event": "refused", "data": json.dumps({"reason": e.reason})}
        return
    except OrchestratorFailed as e:
        yield {"event": "failed", "data": json.dumps({"detail": e.detail})}
        return

    manifest.stage = JobStage.scene_gen
    write_manifest(manifest)
    yield {"event": "stage", "data": json.dumps({"stage": "scene_gen"})}

    try:
        if manifest.scene.generator == "hunyuanworld":
            res = call_scene_hunyuan(manifest.scene.scene_prompt, exhibit_id)
        else:
            res = call_scene_stock(manifest.scene.stock_room_id, exhibit_id)
        manifest.scene.scene_url = res["scene_url"]
        manifest.timing.scene_gen_ms = res["elapsed_ms"]
    except Exception as e:
        yield {"event": "scene_fallback", "data": json.dumps({"reason": str(e)})}
        manifest = orchestrate(user_prompt, exhibit_id=exhibit_id,
force_generator="stock")
        res = call_scene_stock(manifest.scene.stock_room_id, exhibit_id)
        manifest.scene.scene_url = res["scene_url"]
        manifest.timing.scene_gen_ms = res["elapsed_ms"]

    manifest.stage = JobStage.objects_gen
    write_manifest(manifest)
    yield {"event": "stage", "data": json.dumps({"stage": "objects_gen"})}

    calls = {o.id: spawn_object(o.image_prompt, exhibit_id, o.id, resolution=512)
for o in manifest.objects}
    for obj_id, call in calls.items():
```

```
        try:
            res = call.get(timeout=300)
            obj = next(o for o in manifest.objects if o.id == obj_id)
            obj.image_url = res["image_url"]
            obj.glb_url = res["glb_url"]
            manifest.timing.objects_gen_ms[obj_id] = res["elapsed_ms"]
            yield {"event": "object_ready", "data": json.dumps({"object_id":
obj_id})}
        except Exception as e:
            manifest.errors.append(StageError(stage="object_gen", object_id=obj_id,
detail=str(e)))

    manifest.stage = JobStage.ready
    manifest.timing.total_ms = int((time.time() - t_start) * 1000)
    write_manifest(manifest)
    yield {"event": "ready", "data": json.dumps({"exhibit_id": exhibit_id})}
```

- [ ] **Step 7: Rewrite `backend/app/api/exhibits.py`.**

```
"""Exhibits API — real pipeline behind SSE."""
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel, Field
from sse_starlette.sse import EventSourceResponse

from ..routing import run_pipeline
from ..storage.volume import read_manifest

router = APIRouter(prefix="/api", tags=["exhibits"])


class CreateExhibitRequest(BaseModel):
    prompt: str = Field(min_length=1, max_length=500)


@router.post("/exhibits", status_code=202)
async def create_exhibit(req: CreateExhibitRequest) -> dict:
    """Validate prompt; return events_url for SSE subscription."""
    return {
        "exhibit_id": "pending",
        "events_url": f"/api/exhibits/events?prompt={req.prompt}",
    }


@router.get("/exhibits/events")
async def exhibit_events_with_prompt(prompt: str) -> EventSourceResponse:
    """SSE-driven pipeline."""
    return EventSourceResponse(run_pipeline(prompt))


@router.get("/exhibits/{exhibit_id}")
def get_exhibit(exhibit_id: str) -> dict:
```

```
    m = read_manifest(exhibit_id)
    if m is None:
        raise HTTPException(404, detail={"exhibit_id": exhibit_id, "detail": "not
found"})
    return m.model_dump(mode="json")
```

- [ ] **Step 8: Update tests `backend/tests/test_exhibits_api.py`.**

```
"""Exhibits API tests — post-M2."""
from fastapi.testclient import TestClient

from app.main import app

client = TestClient(app)


def test_create_returns_202_with_events_url() -> None:
    r = client.post("/api/exhibits", json={"prompt": "a Victorian study"})
    assert r.status_code == 202
    body = r.json()
    assert "events_url" in body
    assert body["events_url"].startswith("/api/exhibits/events")


def test_rejects_empty_prompt() -> None:
    r = client.post("/api/exhibits", json={"prompt": ""})
    assert r.status_code == 422


def test_unknown_exhibit_404() -> None:
    r = client.get("/api/exhibits/nonexistent_id")
    assert r.status_code == 404
```

- [ ] **Step 9: Run tests.**

```
cd /mnt/3D_Mapping/backend && uv run pytest tests/ -v
```

- [ ] **Step 10: Manual end-to-end smoke.**

```
cd /mnt/3D_Mapping/backend
ANTHROPIC_API_KEY=sk-... uv run uvicorn app.main:app --port 8000 &
sleep 3
curl -N "http://localhost:8000/api/exhibits/events?
prompt=a%20Victorian%20study%20with%20a%20pocket%20watch"
pkill -f 'uvicorn'
```

Expected: SSE stream emits `orchestrating → scene_gen → objects_gen → object_ready (xN) → ready`. Wall clock 5–8 min.

- [ ] **Step 11: Commit.**

```
cd /mnt/3D_Mapping
git add backend/app/modal_client/ backend/app/storage/ backend/app/routing.py
backend/app/api/exhibits.py backend/tests/
git commit -m "feat(api): real pipeline wired — orchestrator + Modal scene/object
gen"
```

Phase 3 — Week 3: Viewer foundation (M3)

By end: real prompt → real manifest → real viewer with walking + click-to-focus (no dissection yet).

Task 18: ExhibitViewer + Scene loader + Pedestals

Files: `frontend/src/viewer/{ExhibitViewer.tsx,Scene.tsx,Pedestals.tsx}`, `frontend/src/viewer/store/exhibitStore.ts`, modify `frontend/src/pages/Exhibit.tsx`.

- [ ] **Step 1: Write `frontend/src/viewer/store/exhibitStore.ts`.**

```
import { create } from "zustand";

import { getExhibit } from "../../api/client";
import type { Manifest } from "../../api/types";

interface ExhibitState {
  manifest: Manifest | null;
  loading: boolean;
  error: string | null;
  load: (id: string) => Promise<void>;
}

export const useExhibitStore = create<ExhibitState>((set) => ({
  manifest: null,
  loading: false,
  error: null,
  load: async (id) => {
    set({ loading: true, error: null });
    try { set({ manifest: await getExhibit(id), loading: false }); }
    catch (e) { set({ error: String(e), loading: false }); }
  },
}));
```

- [ ] **Step 2: Write `frontend/src/viewer/ExhibitViewer.tsx`** (single file with three.js setup + child mounts).

```
import { useEffect, useRef } from "react";
import * as THREE from "three";
```

```
import type { Manifest } from "../api/types";
import { loadScene } from "./Scene";
import { mountPedestals } from "./Pedestals";

interface Props { manifest: Manifest; apiBase: string; }

export default function ExhibitViewer({ manifest, apiBase }: Props) {
  const canvasRef = useRef<HTMLCanvasElement | null>(null);

  useEffect(() => {
    if (!canvasRef.current) return;
    const canvas = canvasRef.current;

    const renderer = new THREE.WebGLRenderer({ canvas, antialias: true });
    renderer.setSize(window.innerWidth, window.innerHeight);
    renderer.setPixelRatio(window.devicePixelRatio);
    renderer.localClippingEnabled = true;

    const scene = new THREE.Scene();
    scene.background = new THREE.Color(manifest.scene.ambient_light.color);
    scene.add(new THREE.AmbientLight(
      manifest.scene.ambient_light.color,
      manifest.scene.ambient_light.intensity,
    ));

    const camera = new THREE.PerspectiveCamera(60, window.innerWidth /
window.innerHeight, 0.05, 100);
    const [cx, cy, cz] = manifest.spline.control_points[0] as [number, number,
number];
    camera.position.set(cx, cy, cz);

    const sceneObjects = loadScene(manifest.scene, scene, apiBase);
    const pedestals = mountPedestals(manifest, scene, apiBase);

    let raf = 0;
    const loop = () => { raf = requestAnimationFrame(loop); renderer.render(scene,
camera); };
    loop();

    const onResize = () => {
      camera.aspect = window.innerWidth / window.innerHeight;
      camera.updateProjectionMatrix();
      renderer.setSize(window.innerWidth, window.innerHeight);
    };
    window.addEventListener("resize", onResize);

    return () => {
      cancelAnimationFrame(raf);
      window.removeEventListener("resize", onResize);
      sceneObjects.dispose();
      pedestals.dispose();
      renderer.dispose();
```

```
    };
  }, [manifest, apiBase]);

  return <canvas ref={canvasRef} className="block" />;
}
```

- [ ] **Step 3: Write `frontend/src/viewer/Scene.tsx`.**

```
import * as THREE from "three";
import { GLTFLoader } from "three/addons/loaders/GLTFLoader.js";

import type { Scene as SceneSpec } from "../api/types";

export function loadScene(spec: SceneSpec, threeScene: THREE.Scene, apiBase: string)
{
  const loader = new GLTFLoader();
  let loaded: THREE.Object3D | null = null;
  loader.load(
    apiBase + spec.scene_url,
    (gltf) => { loaded = gltf.scene; threeScene.add(gltf.scene); },
    undefined,
    (err) => console.error("Scene load failed:", err),
  );
  return {
    dispose: () => { if (loaded) threeScene.remove(loaded); },
  };
}
```

- [ ] **Step 4: Write `frontend/src/viewer/Pedestals.tsx`.**

```
import * as THREE from "three";
import { GLTFLoader } from "three/addons/loaders/GLTFLoader.js";

import type { Manifest } from "../api/types";

export function mountPedestals(manifest: Manifest, threeScene: THREE.Scene, apiBase:
string) {
  const group = new THREE.Group();
  group.name = "pedestals";
  const pedGeom = new THREE.CylinderGeometry(0.25, 0.3, 1.0, 32);
  const pedMat = new THREE.MeshStandardMaterial({ color: 0x202020, roughness: 0.5
});
  const loader = new GLTFLoader();

  for (const ped of manifest.pedestals) {
    const mesh = new THREE.Mesh(pedGeom, pedMat);
    const [x, , z] = ped.position;
    mesh.position.set(x, ped.height / 2, z);
    mesh.name = `pedestal:${ped.id}`;
    group.add(mesh);
```

```
    const obj = manifest.objects.find((o) => o.id === ped.object_id);
    if (obj?.glb_url) {
      loader.load(apiBase + obj.glb_url, (gltf) => {
        gltf.scene.position.set(x, ped.height, z);
        gltf.scene.scale.setScalar(obj.scale);
        gltf.scene.rotation.y = (obj.rotation_y_deg * Math.PI) / 180;
        gltf.scene.name = `object:${obj.id}`;
        group.add(gltf.scene);
      });
    }
  }

  threeScene.add(group);
  return { dispose: () => threeScene.remove(group) };
}
```

- [ ] **Step 5: Update `frontend/src/pages/Exhibit.tsx`.**

```
import { useEffect } from "react";
import { useParams } from "react-router-dom";

import ExhibitViewer from "../viewer/ExhibitViewer";
import { useExhibitStore } from "../viewer/store/exhibitStore";

const API_BASE = import.meta.env.VITE_API_BASE ?? "http://localhost:8000";

export default function Exhibit() {
  const { id } = useParams<{ id: string }>();
  const { manifest, loading, error, load } = useExhibitStore();

  useEffect(() => { if (id) load(id); }, [id, load]);

  if (loading) return <main className="p-8">Loading…</main>;
  if (error) return <main className="p-8 text-red-400">{error}</main>;
  if (!manifest) return <main className="p-8">No exhibit.</main>;

  return <ExhibitViewer manifest={manifest} apiBase={API_BASE} />;
}
```

- [ ] **Step 6: Smoke test** with a real exhibit_id from Phase 2.
- [ ] **Step 7: Commit.**

```
cd /mnt/3D_Mapping
git add frontend/src/viewer/ frontend/src/pages/Exhibit.tsx
git commit -m "feat(viewer): three.js root + scene loader + pedestal placement"
```

Task 19: Catmull-Rom Spline.ts (TDD)

Files: `frontend/src/viewer/locomotion/Spline.ts`, `Spline.test.ts`, modify `frontend/vite.config.ts` for vitest.

- [ ] **Step 1: Configure vitest in `frontend/vite.config.ts`.**

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  test: { environment: "jsdom" },
});
```

(Add `/// <reference types="vitest" />` at top if needed.)

- [ ] **Step 2: Write failing test `frontend/src/viewer/locomotion/Spline.test.ts`.**

```
import { describe, expect, it } from "vitest";

import { CatmullRomSpline } from "./Spline";

const points: [number, number, number][] = [
  [0, 0, 0], [1, 0, 0], [2, 1, 0], [3, 1, 0],
];

describe("CatmullRomSpline", () => {
  it("returns first point at t=0", () => {
    const s = new CatmullRomSpline(points);
    expect(s.position(0)).toEqual([0, 0, 0]);
  });

  it("returns last point at t=1", () => {
    const s = new CatmullRomSpline(points);
    expect(s.position(1)).toEqual([3, 1, 0]);
  });

  it("interpolates monotonically in x", () => {
    const s = new CatmullRomSpline(points);
    expect(s.position(0.5)[0]).toBeGreaterThan(s.position(0.25)[0]);
  });

  it("rejects fewer than 2 points", () => {
    expect(() => new CatmullRomSpline([[0, 0, 0]])).toThrow();
  });
});
```

- [ ] **Step 3: Run, verify fail.**

```
cd /mnt/3D_Mapping/frontend && pnpm vitest run src/viewer/locomotion/Spline.test.ts
```

- [ ] **Step 4: Implement `Spline.ts`.**

```
type Point3 = [number, number, number];

export class CatmullRomSpline {
  constructor(private points: Point3[]) {
    if (points.length < 2) throw new Error("Need >= 2 points");
  }

  position(t: number): Point3 {
    const n = this.points.length;
    if (n === 2) return this.lerp(this.points[0], this.points[1], t);
    const seg = Math.min(Math.floor(t * (n - 1)), n - 2);
    const local = t * (n - 1) - seg;
    const p0 = this.points[Math.max(0, seg - 1)];
    const p1 = this.points[seg];
    const p2 = this.points[seg + 1];
    const p3 = this.points[Math.min(n - 1, seg + 2)];
    return this.catmullRom(p0, p1, p2, p3, local);
  }

  tangent(t: number, eps = 1e-4): Point3 {
    const a = this.position(Math.max(0, t - eps));
    const b = this.position(Math.min(1, t + eps));
    return [b[0] - a[0], b[1] - a[1], b[2] - a[2]];
  }

  private lerp(a: Point3, b: Point3, t: number): Point3 {
    return [a[0] + (b[0] - a[0]) * t, a[1] + (b[1] - a[1]) * t, a[2] + (b[2] - a[2])
* t];
  }

  private catmullRom(p0: Point3, p1: Point3, p2: Point3, p3: Point3, t: number):
Point3 {
    const t2 = t * t;
    const t3 = t2 * t;
    const out: Point3 = [0, 0, 0];
    for (let i = 0; i < 3; i++) {
      out[i] = 0.5 * (
        2 * p1[i] +
        (-p0[i] + p2[i]) * t +
        (2 * p0[i] - 5 * p1[i] + 4 * p2[i] - p3[i]) * t2 +
        (-p0[i] + 3 * p1[i] - 3 * p2[i] + p3[i]) * t3
      );
    }
    return out;
  }
}
```

- [ ] **Step 5: Run tests, verify pass.**
- [ ] **Step 6: Commit.**

```
cd /mnt/3D_Mapping
git add frontend/vite.config.ts frontend/src/viewer/locomotion/Spline.ts
frontend/src/viewer/locomotion/Spline.test.ts
git commit -m "feat(locomotion): Catmull-Rom spline (TDD)"
```

Task 20: RailController + locomotionStore

Files: `frontend/src/viewer/store/locomotionStore.ts`,
`frontend/src/viewer/locomotion/RailController.ts`.

- [ ] **Step 1: Write `locomotionStore.ts`.**

```
import { create } from "zustand";

interface LocomotionState {
  mode: "walking" | "focused";
  currentStation: number;
  inTransition: boolean;
  splineT: number;
  setMode: (m: "walking" | "focused") => void;
  setStation: (i: number) => void;
  setSplineT: (t: number) => void;
  setTransitioning: (b: boolean) => void;
}

export const useLocomotionStore = create<LocomotionState>((set) => ({
  mode: "walking",
  currentStation: 0,
  inTransition: false,
  splineT: 0,
  setMode: (m) => set({ mode: m }),
  setStation: (i) => set({ currentStation: i }),
  setSplineT: (t) => set({ splineT: t }),
  setTransitioning: (b) => set({ inTransition: b }),
}));
```

- [ ] **Step 2: Write `RailController.ts`.**

```
import * as THREE from "three";

import type { Manifest } from "../../api/types";
import { useLocomotionStore } from "../store/locomotionStore";
import { CatmullRomSpline } from "./Spline";
```

```ts
export class RailController {
  private spline: CatmullRomSpline;
  private tweening = false;
  private tweenStart = 0;
  private tweenFrom = 0;
  private tweenTo = 0;
  private tweenDuration = 1;

  constructor(private camera: THREE.PerspectiveCamera, private manifest: Manifest) {
    this.spline = new CatmullRomSpline(manifest.spline.control_points as [number,
number, number][]);
    const t0 = manifest.spline.stations[0].t;
    useLocomotionStore.getState().setSplineT(t0);
    this.applyPosition(t0);
  }

  advance(): void {
    const cur = useLocomotionStore.getState().currentStation;
    const next = Math.min(this.manifest.spline.stations.length - 1, cur + 1);
    if (next !== cur) this.tweenToStation(next);
  }

  retreat(): void {
    const cur = useLocomotionStore.getState().currentStation;
    const prev = Math.max(0, cur - 1);
    if (prev !== cur) this.tweenToStation(prev);
  }

  private tweenToStation(idx: number): void {
    this.tweenFrom = useLocomotionStore.getState().splineT;
    this.tweenTo = this.manifest.spline.stations[idx].t;
    this.tweenStart = performance.now();
    this.tweenDuration = this.manifest.spline.travel_time_s * 1000;
    this.tweening = true;
    useLocomotionStore.getState().setTransitioning(true);
    useLocomotionStore.getState().setStation(idx);
  }

  update(): void {
    if (!this.tweening) return;
    const progress = Math.min(1, (performance.now() - this.tweenStart) /
this.tweenDuration);
    const eased = progress < 0.5
      ? 4 * progress ** 3
      : 1 - (-2 * progress + 2) ** 3 / 2;
    const t = this.tweenFrom + (this.tweenTo - this.tweenFrom) * eased;
    useLocomotionStore.getState().setSplineT(t);
    this.applyPosition(t);
    if (progress >= 1) {
      this.tweening = false;
      useLocomotionStore.getState().setTransitioning(false);
    }
```

```
  }

  private applyPosition(t: number): void {
    const [x, y, z] = this.spline.position(t);
    this.camera.position.set(x, y, z);
  }
}
```

- [ ] **Step 3: Mount in `ExhibitViewer.tsx` useEffect.**

```
import { RailController } from "./locomotion/RailController";
import { useLocomotionStore } from "./store/locomotionStore";

// ... inside useEffect, after creating `camera`:
const rail = new RailController(camera, manifest);

const onKey = (e: KeyboardEvent) => {
  if (useLocomotionStore.getState().mode !== "walking") return;
  if (e.key === "ArrowRight" || e.key === " " || e.key === "ArrowDown")
rail.advance();
  if (e.key === "ArrowLeft" || e.key === "ArrowUp") rail.retreat();
};
window.addEventListener("keydown", onKey);

// in loop:
rail.update();

// cleanup:
window.removeEventListener("keydown", onKey);
```

- [ ] **Step 4: Smoke test.** Press `→`, watch camera glide.
- [ ] **Step 5: Commit.**

```
cd /mnt/3D_Mapping
git add frontend/src/viewer/locomotion/RailController.ts
frontend/src/viewer/store/locomotionStore.ts frontend/src/viewer/ExhibitViewer.tsx
git commit -m "feat(locomotion): RailController station-to-station tween"
```

Task 21: MouseLook + yaw align

Files: `frontend/src/viewer/locomotion/MouseLook.ts`.

- [ ] **Step 1: Write `MouseLook.ts`.**

```
import * as THREE from "three";

export class MouseLook {
```

```
  private yaw = 0;
  private pitch = 0;
  private active = false;

  constructor(private camera: THREE.PerspectiveCamera, private element: HTMLElement)
{
    element.addEventListener("click", this.onClick);
    document.addEventListener("pointerlockchange", this.onLockChange);
    document.addEventListener("mousemove", this.onMove);
  }

  setYawDeg(deg: number): void {
    this.yaw = (deg * Math.PI) / 180;
    this.apply();
  }

  dispose(): void {
    this.element.removeEventListener("click", this.onClick);
    document.removeEventListener("pointerlockchange", this.onLockChange);
    document.removeEventListener("mousemove", this.onMove);
  }

  private onClick = () => this.element.requestPointerLock();
  private onLockChange = () => { this.active = document.pointerLockElement ===
this.element; };
  private onMove = (e: MouseEvent) => {
    if (!this.active) return;
    const sens = 0.002;
    this.yaw -= e.movementX * sens;
    this.pitch -= e.movementY * sens;
    this.pitch = Math.max(-Math.PI / 2 + 0.01, Math.min(Math.PI / 2 - 0.01,
this.pitch));
    this.apply();
  };

  private apply(): void {
    this.camera.rotation.order = "YXZ";
    this.camera.rotation.y = this.yaw;
    this.camera.rotation.x = this.pitch;
  }
}
```

- ☐ **Step 2: Mount in `ExhibitViewer.tsx`.**

```
import { MouseLook } from "./locomotion/MouseLook";

// in useEffect:
const mouseLook = new MouseLook(camera, canvas);
// initial yaw from first station
mouseLook.setYawDeg(manifest.spline.stations[0].preferred_yaw_deg);
```

```
// cleanup:
mouseLook.dispose();
```

(For yaw alignment on station arrival: extend RailController to call `mouseLook.setYawDeg()` on tween completion. Or simpler: poll `inTransition` + `currentStation` and align on arrival.)

- [ ] **Step 3: Commit.**

```
cd /mnt/3D_Mapping
git add frontend/src/viewer/locomotion/MouseLook.ts
frontend/src/viewer/ExhibitViewer.tsx
git commit -m "feat(locomotion): MouseLook with pointer lock"
```

Task 22: HUD elements

Files: `frontend/src/viewer/hud/{StationIndicator,ProgressBar,HintChip}.tsx`, modify `frontend/src/pages/Exhibit.tsx`.

- [ ] **Step 1: Write `StationIndicator.tsx`.**

```
import { useExhibitStore } from "../store/exhibitStore";
import { useLocomotionStore } from "../store/locomotionStore";

export default function StationIndicator() {
  const manifest = useExhibitStore((s) => s.manifest);
  const current = useLocomotionStore((s) => s.currentStation);
  if (!manifest) return null;
  const st = manifest.spline.stations[current];
  return (
    <div className="absolute top-4 left-1/2 -translate-x-1/2 bg-black/40 px-4 py-2
rounded text-sm">
      Station {current + 1} of {manifest.spline.stations.length} — {st.label}
    </div>
  );
}
```

- [ ] **Step 2: Write `ProgressBar.tsx`.**

```
import { useLocomotionStore } from "../store/locomotionStore";

export default function ProgressBar() {
  const t = useLocomotionStore((s) => s.splineT);
  return (
    <div className="absolute bottom-6 left-1/2 -translate-x-1/2 w-64 h-1 bg-white/10
rounded-full overflow-hidden">
      <div className="h-full bg-amber-400 transition-all duration-150" style={{
width: `${t * 100}%` }} />
    </div>
```

```
  );
}
```

- [ ] **Step 3: Write `HintChip.tsx`.**

```
import { useEffect, useState } from "react";

export default function HintChip() {
  const [visible, setVisible] = useState(true);
  useEffect(() => {
    const onKey = () => setVisible(false);
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown", onKey);
  }, []);
  if (!visible) return null;
  return (
    <div className="absolute bottom-4 right-4 bg-black/50 px-3 py-2 rounded text-xs
text-white/80">
      → next | ← back | click to inspect
    </div>
  );
}
```

- [ ] **Step 4: Mount in `Exhibit.tsx`.**

```
import StationIndicator from "../viewer/hud/StationIndicator";
import ProgressBar from "../viewer/hud/ProgressBar";
import HintChip from "../viewer/hud/HintChip";

// in render:
return (
  <>
    <ExhibitViewer manifest={manifest} apiBase={API_BASE} />
    <StationIndicator />
    <ProgressBar />
    <HintChip />
  </>
);
```

- [ ] **Step 5: Commit.**

```
cd /mnt/3D_Mapping
git add frontend/src/viewer/hud/ frontend/src/pages/Exhibit.tsx
git commit -m "feat(viewer): HUD station indicator + progress + hint"
```

Task 23: FocusController + ModeManager + click-to-focus

Files: `frontend/src/viewer/locomotion/{FocusController,ModeManager}.ts`, modify `frontend/src/viewer/ExhibitViewer.tsx`.

- [ ] **Step 1: Write `FocusController.ts`.**

```
import * as THREE from "three";

import type { ObjectSpec, PedestalSpec } from "../../api/types";

export class FocusController {
  private originalPos = new THREE.Vector3();
  private originalQuat = new THREE.Quaternion();
  private targetPos = new THREE.Vector3();
  private targetQuat = new THREE.Quaternion();
  private tweenStart = 0;
  private tweenDuration = 1000;
  private active = false;

  constructor(private camera: THREE.PerspectiveCamera) {}

  enter(_object: ObjectSpec, _pedestal: PedestalSpec, objectMesh: THREE.Object3D):
void {
    this.originalPos.copy(this.camera.position);
    this.originalQuat.copy(this.camera.quaternion);

    const center = new THREE.Vector3();
    new THREE.Box3().setFromObject(objectMesh).getCenter(center);
    const forward = new THREE.Vector3();
    this.camera.getWorldDirection(forward);
    this.targetPos.copy(center).addScaledVector(forward, -0.6);

    const tmp = this.camera.clone();
    tmp.position.copy(this.targetPos);
    tmp.lookAt(center);
    this.targetQuat.copy(tmp.quaternion);

    this.tweenStart = performance.now();
    this.active = true;
  }

  exit(): void {
    [this.targetPos, this.originalPos] = [this.originalPos, this.targetPos];
    [this.targetQuat, this.originalQuat] = [this.originalQuat, this.targetQuat];
    this.tweenStart = performance.now();
    this.active = true;
  }

  update(): void {
    if (!this.active) return;
    const t = Math.min(1, (performance.now() - this.tweenStart) /
this.tweenDuration);
```

```
    const eased = t * t * (3 - 2 * t);
    this.camera.position.lerpVectors(this.originalPos, this.targetPos, eased);
    this.camera.quaternion.copy(this.originalQuat).slerp(this.targetQuat, eased);
    if (t >= 1) this.active = false;
  }
}
```

- [ ] **Step 2: Write `ModeManager.ts`.**

```
import * as THREE from "three";

import type { ObjectSpec, PedestalSpec } from "../../api/types";
import { useLocomotionStore } from "../store/locomotionStore";
import type { FocusController } from "./FocusController";

export class ModeManager {
  constructor(private focus: FocusController) {}

  enterFocus(o: ObjectSpec, p: PedestalSpec, mesh: THREE.Object3D): void {
    useLocomotionStore.getState().setMode("focused");
    this.focus.enter(o, p, mesh);
  }

  exitFocus(): void {
    this.focus.exit();
    useLocomotionStore.getState().setMode("walking");
  }
}
```

- [ ] **Step 3: Wire click + Esc handlers in `ExhibitViewer.tsx`.**

```
import { FocusController } from "./locomotion/FocusController";
import { ModeManager } from "./locomotion/ModeManager";

// in useEffect:
const focus = new FocusController(camera);
const modeMgr = new ModeManager(focus);

const raycaster = new THREE.Raycaster();
const onClick = (e: MouseEvent) => {
  if (useLocomotionStore.getState().mode !== "walking") return;
  const ndc = new THREE.Vector2(
    (e.clientX / window.innerWidth) * 2 - 1,
    -(e.clientY / window.innerHeight) * 2 + 1,
  );
  raycaster.setFromCamera(ndc, camera);
  const hits = raycaster.intersectObjects(scene.children, true);
  for (const hit of hits) {
    let cur: THREE.Object3D | null = hit.object;
    while (cur) {
```

```
      const m = cur.name.match(/^(?:pedestal|object):(.+)$/);
      if (m) {
        const id = m[1];
        const obj = manifest.objects.find((o) => o.id === id);
        const ped = obj && manifest.pedestals.find((p) => p.object_id === obj.id);
        const mesh = obj && scene.getObjectByName(`object:${obj.id}`);
        if (obj && ped && mesh) modeMgr.enterFocus(obj, ped, mesh);
        return;
      }
      cur = cur.parent;
    }
  }
};

const onEsc = (e: KeyboardEvent) => {
  if (e.key === "Escape" && useLocomotionStore.getState().mode === "focused")
modeMgr.exitFocus();
};

canvas.addEventListener("click", onClick);
window.addEventListener("keydown", onEsc);

// in loop, also call focus.update();
// cleanup: remove listeners
```

- [ ] **Step 4: Smoke test** — click pedestal → fly in; Esc → fly out.
- [ ] **Step 5: Commit.**

```
cd /mnt/3D_Mapping
git add frontend/src/viewer/locomotion/
git commit -m "feat(locomotion): focus mode (click pedestal -> fly to hero pose)"
```

Phase 4 — Week 4: Dissection (M4)

Task 24: DissectionStore + ClippingPlane

Files: `frontend/src/viewer/dissection/{DissectionStore,ClippingPlane}.ts`.

- [ ] **Step 1: Write `DissectionStore.ts`.**

```
import { create } from "zustand";

interface DissectionState {
  focusedObjectId: string | null;
  clipAxis: "x" | "y" | "z";
  clipT: number;
  explodeT: number;
```

```
  setFocused: (id: string | null) => void;
  setClip: (axis: "x" | "y" | "z", t: number) => void;
  setExplode: (t: number) => void;
  reset: () => void;
}

export const useDissectionStore = create<DissectionState>((set) => ({
  focusedObjectId: null,
  clipAxis: "y",
  clipT: 0,
  explodeT: 0,
  setFocused: (id) => set({ focusedObjectId: id, clipT: 0, explodeT: 0 }),
  setClip: (axis, t) => set({ clipAxis: axis, clipT: t }),
  setExplode: (t) => set({ explodeT: t }),
  reset: () => set({ clipT: 0, explodeT: 0 }),
}));
```

- [ ] **Step 2: Write `ClippingPlane.ts`.**

```
import * as THREE from "three";

export class ClippingPlane {
  readonly plane = new THREE.Plane();

  setAxis(axis: "x" | "y" | "z"): void {
    if (axis === "x") this.plane.normal.set(-1, 0, 0);
    if (axis === "y") this.plane.normal.set(0, -1, 0);
    if (axis === "z") this.plane.normal.set(0, 0, -1);
  }

  setOffset(t: number, bboxHalfExtent: number): void {
    this.plane.constant = (t - 0.5) * 2 * bboxHalfExtent;
  }
}
```

- [ ] **Step 3: Wire into focused-object materials on focus enter.**

In ModeManager or FocusController's enter, after capturing the object mesh:

```
const bbox = new THREE.Box3().setFromObject(objectMesh);
const halfExtent = (bbox.max.y - bbox.min.y) / 2;  // y for default axis
const clipping = new ClippingPlane();
clipping.setAxis("y");
clipping.setOffset(0, halfExtent);

objectMesh.traverse((o) => {
  const m = (o as THREE.Mesh).material as THREE.MeshStandardMaterial | undefined;
  if (m) m.clippingPlanes = [clipping.plane];
});

// Subscribe to DissectionStore to update offset/axis live:
```

```
useDissectionStore.subscribe((s) => {
  clipping.setAxis(s.clipAxis);
  clipping.setOffset(s.clipT, halfExtent);
});
```

- [ ] **Step 4: Commit.**

```
cd /mnt/3D_Mapping
git add frontend/src/viewer/dissection/
git commit -m "feat(dissection): clipping plane + dissection store"
```

Task 25: Capped clipping (stencil-based)

Files: `frontend/src/viewer/dissection/CappedClipMaterial.ts`.

Port from Spike C; productionize.

- [ ] **Step 1: Write `CappedClipMaterial.ts`.**

```
import * as THREE from "three";

export class CappedClipping {
  private capMesh: THREE.Mesh;
  private active = false;

  constructor(private scene: THREE.Scene) {
    const geo = new THREE.PlaneGeometry(10, 10);
    const mat = new THREE.MeshBasicMaterial({
      color: 0x555555,
      stencilWrite: true, stencilRef: 1, stencilFunc: THREE.EqualStencilFunc,
      stencilFail: THREE.KeepStencilOp, stencilZFail: THREE.KeepStencilOp,
stencilZPass: THREE.KeepStencilOp,
    });
    this.capMesh = new THREE.Mesh(geo, mat);
    this.capMesh.renderOrder = 1;
    this.capMesh.visible = false;
    scene.add(this.capMesh);
  }

  attach(object: THREE.Object3D, plane: THREE.Plane, capColor: THREE.Color): void {
    object.traverse((o) => {
      const mesh = o as THREE.Mesh;
      if (!mesh.isMesh) return;
      const m = mesh.material as THREE.MeshStandardMaterial;
      m.clippingPlanes = [plane];
      m.stencilWrite = true;
      m.stencilRef = 1;
      m.stencilFunc = THREE.AlwaysStencilFunc;
      m.stencilZPass = THREE.ReplaceStencilOp;
      m.side = THREE.DoubleSide;
```

```
      m.needsUpdate = true;
    });
    (this.capMesh.material as THREE.MeshBasicMaterial).color = capColor;
    this.capMesh.visible = true;
    this.active = true;
  }

  update(plane: THREE.Plane): void {
    if (!this.active) return;
    this.capMesh.position.set(0, 0, 0).addScaledVector(plane.normal, -
plane.constant);
    const lookTarget = new THREE.Vector3().addVectors(this.capMesh.position,
plane.normal);
    this.capMesh.lookAt(lookTarget);
  }

  detach(): void {
    this.capMesh.visible = false;
    this.active = false;
  }
}
```

- [ ] **Step 2: Sample first mesh's albedo on focus enter** to pass as `capColor`.
- [ ] **Step 3: Update CappedClipping on every frame in loop.**
- [ ] **Step 4: Smoke test.** Focus on watch → slide clip → cap is visible.
- [ ] **Step 5: Commit.**

```
cd /mnt/3D_Mapping
git add frontend/src/viewer/dissection/CappedClipMaterial.ts
git commit -m "feat(dissection): capped cross-section via stencil"
```

Task 26: Exploded view

Files: `frontend/src/viewer/dissection/ExplodedView.ts`.

- [ ] **Step 1: Write.**

```
import * as THREE from "three";

interface Part { mesh: THREE.Object3D; originalPos: THREE.Vector3; outwardDir:
THREE.Vector3; }

export class ExplodedView {
  private parts: Part[] = [];
  private maxDistance = 0;
```

```
  attach(root: THREE.Object3D): boolean {
    const children = root.children.filter((c) => (c as THREE.Mesh).isMesh ||
c.children.length > 0);
    if (children.length < 2) return false;

    const center = new THREE.Vector3();
    new THREE.Box3().setFromObject(root).getCenter(center);
    this.parts = children.map((mesh) => {
      const centroid = new THREE.Vector3();
      new THREE.Box3().setFromObject(mesh).getCenter(centroid);
      const dir = new THREE.Vector3().subVectors(centroid, center).normalize();
      const d = centroid.distanceTo(center);
      this.maxDistance = Math.max(this.maxDistance, d);
      return { mesh, originalPos: mesh.position.clone(), outwardDir: dir };
    });
    return true;
  }

  setT(t: number, scale = 1.5): void {
    for (const p of this.parts) {
      const offset = p.outwardDir.clone().multiplyScalar(t * scale *
this.maxDistance);
      p.mesh.position.copy(p.originalPos).add(offset);
    }
  }

  detach(): void {
    for (const p of this.parts) p.mesh.position.copy(p.originalPos);
    this.parts = [];
  }
}
```

- [ ] **Step 2: Wire into focus enter (attach) + DissectionStore subscription (setT) + focus exit (detach).**
- [ ] **Step 3: Commit.**

```
cd /mnt/3D_Mapping
git add frontend/src/viewer/dissection/ExplodedView.ts
git commit -m "feat(dissection): exploded view for multi-component GLBs"
```

Task 27: DissectionPanel React UI

Files: `frontend/src/viewer/dissection/DissectionPanel.tsx`, modify `frontend/src/pages/Exhibit.tsx`.

- [ ] **Step 1: Write `DissectionPanel.tsx`.**

```
import { useDissectionStore } from "./DissectionStore";
import { useExhibitStore } from "../store/exhibitStore";
import { useLocomotionStore } from "../store/locomotionStore";

export default function DissectionPanel({ onClose }: { onClose: () => void }) {
  const manifest = useExhibitStore((s) => s.manifest);
  const focusedObjectId = useDissectionStore((s) => s.focusedObjectId);
  const clipAxis = useDissectionStore((s) => s.clipAxis);
  const clipT = useDissectionStore((s) => s.clipT);
  const explodeT = useDissectionStore((s) => s.explodeT);
  const setClip = useDissectionStore((s) => s.setClip);
  const setExplode = useDissectionStore((s) => s.setExplode);
  const reset = useDissectionStore((s) => s.reset);
  const mode = useLocomotionStore((s) => s.mode);

  if (mode !== "focused" || !manifest || !focusedObjectId) return null;
  const obj = manifest.objects.find((o) => o.id === focusedObjectId);
  if (!obj) return null;

  return (
    <aside className="absolute top-4 right-4 w-80 bg-black/70 p-4 rounded text-white
space-y-3">
      <h2 className="text-lg font-bold">{obj.label}</h2>
      <p className="text-sm opacity-80">{obj.placard_description}</p>
      <div>
        <label className="text-xs block">Cut along</label>
        <select
          className="bg-black/60 px-2 py-1 rounded mt-1"
          value={clipAxis}
          onChange={(e) => setClip(e.target.value as "x" | "y" | "z", clipT)}
        >
          <option value="x">X</option><option value="y">Y</option><option
value="z">Z</option>
        </select>
      </div>
      <div>
        <label className="text-xs block">Cross-section position</label>
        <input type="range" min={0} max={1} step={0.01} value={clipT}
          onChange={(e) => setClip(clipAxis, parseFloat(e.target.value))}
          className="w-full" />
        <span className="text-xs opacity-60">{Math.round(clipT * 100)}%</span>
      </div>
      <div>
        <label className="text-xs block">Exploded view</label>
        <input type="range" min={0} max={1} step={0.01} value={explodeT}
          onChange={(e) => setExplode(parseFloat(e.target.value))}
          className="w-full" />
      </div>
      <div className="flex gap-2">
        <button className="px-3 py-1 bg-white/10 rounded" onClick=
{reset}>Reset</button>
```

```
        <button className="px-3 py-1 bg-white/10 rounded" onClick={onClose}>Back to
exhibit</button>
      </div>
    </aside>
  );
}
```

- [ ] **Step 2: Mount in `Exhibit.tsx`.**

```
import DissectionPanel from "../viewer/dissection/DissectionPanel";

// add to render:
<DissectionPanel onClose={() => {
  // trigger ModeManager.exitFocus — easiest via a custom event or store callback
  window.dispatchEvent(new CustomEvent("exit-focus"));
}} />
```

In ExhibitViewer's useEffect, listen for `exit-focus` and call `modeMgr.exitFocus()`.

- [ ] **Step 3: Commit.**

```
cd /mnt/3D_Mapping
git add frontend/src/viewer/dissection/DissectionPanel.tsx
frontend/src/pages/Exhibit.tsx frontend/src/viewer/ExhibitViewer.tsx
git commit -m "feat(dissection): React side panel UI"
```

Phase 5 — Week 5: Curation, polish (M5)

Task 28: Three curated exhibits (hand-tuned)

- [ ] **Step 1: Generate each.**

```
curl -N "http://localhost:8000/api/exhibits/events?prompt=An%20anatomy%20exhibit"
# capture exhibit_id from the "ready" event
```

Repeat for Horology and Botany.

- [ ] **Step 2: QA each in browser.** Note: bad scale, bad placement, ugly object, missing GLB.
- [ ] **Step 3: Iterate orchestrator system prompt + few-shot fixtures.** Budget 5–10 iterations per exhibit. Each iteration costs ~$0.50.
- [ ] **Step 4: Pin the accepted exhibits.**

Download each manifest:

```
curl http://localhost:8000/api/exhibits/<anatomy_id> >
frontend/public/curated/anatomy.json
curl http://localhost:8000/api/exhibits/<horology_id> >
frontend/public/curated/horology.json
curl http://localhost:8000/api/exhibits/<botany_id> >
frontend/public/curated/botany.json
```

Also copy the artifact dirs (`scene/` , `objects/`) into `frontend/public/curated/<id>/` .

- [ ] **Step 5: Add curated route to backend.**

```
# in backend/app/api/exhibits.py
@router.get("/exhibits/curated_{name}")
def curated(name: str) -> dict:
    p = Path("frontend/public/curated") / f"{name}.json"
    if not p.exists():
        raise HTTPException(404)
    return json.loads(p.read_text())
```

- [ ] **Step 6: Commit.**

```
cd /mnt/3D_Mapping
git add frontend/public/curated/ backend/app/orchestrator/prompts.py
backend/tests/fixtures/orchestrator/few_shot/ backend/app/api/exhibits.py
git commit -m "feat(curate): three hand-tuned launch exhibits"
```

Task 29: Landing page (hero + gallery)

Files: modify `frontend/src/pages/Landing.tsx` .

- [ ] **Step 1: Write.**

```
import { Link } from "react-router-dom";

const PRESETS = [
  { id: "anatomy", title: "The Anatomy Theatre", subtitle: "Heart, brain, skeleton"
},
  { id: "horology", title: "The Horologist's Bench", subtitle: "Three pocket
watches" },
  { id: "botany", title: "The Botanist's Specimens", subtitle: "Fruits, seeds,
plants" },
];

export default function Landing() {
  return (
    <main>
      <section className="text-center py-24 px-8">
        <h1 className="text-5xl font-bold">Interactive AI Museums</h1>
```

```
        <p className="mt-4 text-xl opacity-80">Type a prompt. Step inside. Take it
apart.</p>
        <Link to="/generate" className="mt-8 inline-block bg-amber-500 text-black
px-6 py-3 rounded font-semibold">
          Generate your own →
        </Link>
      </section>
      <section className="px-8 pb-24">
        <h2 className="text-2xl mb-4">Curated exhibits</h2>
        <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
          {PRESETS.map((p) => (
            <Link key={p.id} to={`/exhibit/curated_${p.id}`} className="border
border-white/10 rounded p-4 hover:bg-white/5">
              <h3 className="font-bold">{p.title}</h3>
              <p className="text-sm opacity-70 mt-2">{p.subtitle}</p>
            </Link>
          ))}
        </div>
      </section>
    </main>
  );
}
```

- [ ] **Step 2: Commit.**

```
cd /mnt/3D_Mapping
git add frontend/src/pages/Landing.tsx
git commit -m "feat(landing): hero + curated gallery"
```

Task 30: Generate page (SSE progress)

Files: modify `frontend/src/pages/Generate.tsx`.

- [ ] **Step 1: Write.**

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";

const LABELS: Record<string, string> = {
  orchestrating: "Planning your exhibit…",
  scene_gen: "Building the room…",
  objects_gen: "Generating objects…",
  object_ready: "Crafted an object",
  ready: "Done!",
  refused: "This prompt couldn't be exhibited",
  failed: "Something went wrong",
  scene_fallback: "Using a stock room — AI scene gen failed",
};
```

```
const API_BASE = import.meta.env.VITE_API_BASE ?? "http://localhost:8000";

export default function Generate() {
  const [prompt, setPrompt] = useState("");
  const [stage, setStage] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);
  const navigate = useNavigate();

  const submit = () => {
    setStage("connecting");
    setError(null);
    const es = new EventSource(`${API_BASE}/api/exhibits/events?
prompt=${encodeURIComponent(prompt)}`);
    es.addEventListener("stage", (e) => {
      const d = JSON.parse((e as MessageEvent).data);
      setStage(d.stage);
    });
    es.addEventListener("object_ready", () => setStage("object_ready"));
    es.addEventListener("ready", (e) => {
      const d = JSON.parse((e as MessageEvent).data);
      es.close();
      navigate(`/exhibit/${d.exhibit_id}`);
    });
    es.addEventListener("refused", (e) => {
      const d = JSON.parse((e as MessageEvent).data);
      setError(`Refused: ${d.reason}`);
      es.close();
    });
    es.addEventListener("failed", (e) => {
      const d = JSON.parse((e as MessageEvent).data);
      setError(`Failed: ${d.detail}`);
      es.close();
    });
  };

  return (
    <main className="p-8 max-w-2xl mx-auto">
      <h1 className="text-3xl">Generate an exhibit</h1>
      <textarea
        className="mt-4 w-full bg-black/40 border border-white/20 rounded p-3"
        rows={3}
        placeholder="A Victorian study with a working pocket watch"
        value={prompt}
        onChange={(e) => setPrompt(e.target.value)}
      />
      <button
        className="mt-4 bg-amber-500 text-black px-6 py-2 rounded font-semibold
disabled:opacity-40"
        onClick={submit}
        disabled={stage !== null && stage !== "ready" && stage !== "refused" &&
stage !== "failed"}
      >
```

```
        Generate
      </button>
      {stage && <p className="mt-6 text-lg">{LABELS[stage] ?? stage}</p>}
      {error && <p className="mt-6 text-red-400">{error}</p>}
    </main>
  );
}
```

- [ ] **Step 2: Commit.**

```
cd /mnt/3D_Mapping
git add frontend/src/pages/Generate.tsx
git commit -m "feat(generate): SSE-driven prompt UI"
```

Task 31: Open Graph image generation

Files: `backend/app/api/og.py`, modify `backend/app/main.py`, modify `frontend/index.html`.

- [ ] **Step 1: Write `backend/app/api/og.py`.**

```
"""Open Graph image (1200x630) for share cards."""
from io import BytesIO

from fastapi import APIRouter, HTTPException, Response
from PIL import Image, ImageDraw, ImageFont

from ..storage.volume import read_manifest

router = APIRouter()


@router.get("/og/{exhibit_id}.png")
def og_image(exhibit_id: str) -> Response:
    m = read_manifest(exhibit_id)
    if not m:
        raise HTTPException(404)
    img = Image.new("RGB", (1200, 630), color=(10, 10, 10))
    draw = ImageDraw.Draw(img)
    try:
        title_font =
ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSerif-Bold.ttf", 64)
        desc_font =
ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf", 28)
    except OSError:
        title_font = ImageFont.load_default()
        desc_font = ImageFont.load_default()
    draw.text((60, 220), m.title, font=title_font, fill=(245, 230, 200))
    draw.text((60, 320), m.description[:200], font=desc_font, fill=(180, 180, 180))
    draw.text((60, 560), "Interactive AI Museums", font=desc_font, fill=(140, 140,
```

```
140))
    buf = BytesIO()
    img.save(buf, format="PNG")
    return Response(buf.getvalue(), media_type="image/png")
```

- [ ] **Step 2: Register in `main.py`.**

```
from .api.og import router as og_router
app.include_router(og_router)
```

- [ ] **Step 3: Update `frontend/index.html`** with OG meta tags using the JS-rendered exhibit_id.

(For v1, OG meta is static; for per-exhibit OG, server-side render the meta — defer to v2.)

- [ ] **Step 4: Commit.**

```
cd /mnt/3D_Mapping
git add backend/app/api/og.py backend/app/main.py
git commit -m "feat(share): OG image generation"
```

Task 32: Error boundaries + WebGL recovery + mobile notice

Files: `frontend/src/components/ErrorBoundary.tsx`, `frontend/src/components/MobileNotice.tsx`, modify `frontend/src/pages/Exhibit.tsx`.

- [ ] **Step 1: Write `ErrorBoundary.tsx`.**

```
import React from "react";

interface State { error: Error | null; }

export class ErrorBoundary extends React.Component<{ children: React.ReactNode },
State> {
  state: State = { error: null };
  static getDerivedStateFromError(error: Error): State { return { error }; }
  componentDidCatch(error: Error, info: React.ErrorInfo) { console.error(error,
info); }
  render() {
    if (this.state.error) {
      return (
        <div className="p-8">
          <h1 className="text-2xl">Something went wrong.</h1>
          <p className="mt-4 opacity-70">{this.state.error.message}</p>
          <button onClick={() => window.location.reload()} className="mt-4
underline">Reload</button>
        </div>
      );
    }
    return this.props.children;
```

```
  }
}
```

- [ ] **Step 2: Wrap `<App />` in main.tsx with ErrorBoundary.**
- [ ] **Step 3: Write `MobileNotice.tsx`** and gate viewer mount in `Exhibit.tsx` on `window.innerWidth >= 800`.
- [ ] **Step 4: Add WebGL context-lost handler in `ExhibitViewer.tsx`.**

```
canvas.addEventListener("webglcontextlost", (e) => {
  e.preventDefault();
  alert("WebGL context lost. Reloading.");
  window.location.reload();
});
```

- [ ] **Step 5: Manual cross-browser test** (Chrome, Firefox, Safari, Edge).
- [ ] **Step 6: Commit.**

```
cd /mnt/3D_Mapping
git add frontend/src/
git commit -m "feat(viewer): error boundary + WebGL recovery + mobile notice"
```

Phase 6 — Week 6: Launch (M6)

Task 33: Demo videos

- [ ] **Step 1: Record 3 videos** with OBS or Loom. Save to `marketing/videos/{30s,90s,3min}.mp4`. Embed in Landing.tsx hero.
- [ ] **Step 2: Commit.** (Videos gitignored; document in README.)

Task 34: Production deploy

- [ ] **Step 1: Deploy backend on Modal.**

Write `modal/deploy.py`:

```
"""Deploy FastAPI backend as Modal asgi_app."""
import modal

from .shared import app, base_image

backend_image =
base_image.pip_install_from_pyproject("backend/pyproject.toml").add_local_dir("backend'
```

```
remote_path="/app")


@app.function(image=backend_image, secrets=[modal.Secret.from_name("museums")])
@modal.asgi_app()
def fastapi_app():
    from app.main import app as fastapi
    return fastapi
```

Set Modal secret with `ANTHROPIC_API_KEY`. Deploy:

```
modal deploy modal/deploy.py
```

- [ ] **Step 2: Deploy frontend on Cloudflare Pages.**

```
cd /mnt/3D_Mapping/frontend
echo "VITE_API_BASE=https://<your-modal-app>.modal.run" > .env.production
pnpm build
# Connect Cloudflare Pages to git repo or push dist/ via wrangler
```

- [ ] **Step 3: Domain.** Buy + DNS to Cloudflare. Verify HTTPS.
- [ ] **Step 4: Smoke test from outside.** Generate exhibit → walk → dissect → share URL.
- [ ] **Step 5: Commit deploy artifacts.**

```
cd /mnt/3D_Mapping
git add modal/deploy.py frontend/.env.production
git commit -m "deploy: production wiring"
```

Task 35: Rate limiting + spend caps

- [ ] **Step 1: Per-IP daily cap middleware in FastAPI.**

```
# backend/app/api/middleware.py
from collections import defaultdict
from datetime import date

from fastapi import HTTPException, Request

_counts: dict[tuple[str, date], int] = defaultdict(int)

async def per_ip_cap_middleware(request: Request, call_next, cap: int = 5):
    if request.url.path.startswith("/api/exhibits/events"):
        ip = request.client.host
        key = (ip, date.today())
        if _counts[key] >= cap:
```

```
            raise HTTPException(429, detail="Daily limit reached")
        _counts[key] += 1
    return await call_next(request)
```

Register in `main.py`.

- [ ] **Step 2: Modal dashboard — set daily spend cap = $10.**
- [ ] **Step 3: Commit.**

```
git commit -am "feat(ops): per-IP rate limit + Modal spend cap"
```

Task 36: Analytics (Plausible)

- [ ] **Step 1: Add Plausible script in `frontend/index.html`.**

```
<script defer data-domain="<your-domain>" src="https://plausible.io/js/script.js">
</script>
```

- [ ] **Step 2: Track events from `Generate.tsx`** via `window.plausible('Exhibit Generated')` etc.
- [ ] **Step 3: Commit.**

Task 37: Soft launch

- [ ] **Step 1: Send URL to 5 friends + watch them use it.**
- [ ] **Step 2: Fix top 3 confusions.**
- [ ] **Step 3: Commit fixes.**

Task 38: Public launch

- [ ] **Step 1: Tweet thread.** 5 tweets, 1 video each. Hashtags: #threejs #generative3D #AI.
- [ ] **Step 2: Show HN post.** Title: "Show HN: Interactive AI museums you walk into and take apart". Body: 200 words + link.
- [ ] **Step 3: Reddit posts.** /r/threejs, /r/blender, /r/proceduralgeneration, /r/SideProject.
- [ ] **Step 4: Monitor Modal usage for 48h.** If spend > $20/day, throttle.
- [ ] **Step 5: Write `marketing/launch_post_mortem.md`** at 48h. What worked, what broke, what next.

Self-review

Spec coverage

Spec section	Plan task(s)
§0 summary	1–38 collectively
§1 context	1 (motivation honored)
§2 locked decisions	all tasks honor
§3 architecture	2, 4, 17, 18
§4 orchestrator + manifest	3, 13
§5 rails locomotion	19–23
§6 dissection viewer	24–27
§7 scene gen + fallback	14, 16, 17 (degradation)
§8 object gen	15
§9 module layout	file structure + 2, 5, 13, 17, 18
§10 spike plan	9, 10, 11, 12
§11 milestones	phases 1–6 mirror M0–M6
§12 error handling	17, 32
§13 testing strategy	3, 13, 17, 32
§14 risks	spike + fallback paths
§15 out of scope	respected
§16 open questions	resolved within tasks
§17 acceptance	this plan satisfies the spec

No spec sections uncovered.

Placeholder scan

- Task 9 step 2, Task 10 step 2, Task 14 step 1, Task 15 step 1 contain "CONFIRM" markers where the exact ML library API must be confirmed against the upstream repos before running. These are not placeholders — they are explicit, scoped research steps with a clear preceding "read the docs" task. Acceptable.
- Task 16 step 1: "source from Polyhaven / Sketchfab" — research step, not a code placeholder.
- Task 28: "iterate orchestrator prompts" — bounded iteration with cost budget, not a placeholder.
- No "TBD", "TODO", "implement later", "add appropriate error handling" anywhere.

Type consistency

- `Manifest` shape consistent between `backend/app/schemas.py`, `frontend/src/api/types.ts`, and

`backend/tests/fixtures/orchestrator/few_shot/*.json`.

- `JobStage` values match across backend + frontend.
- `CatmullRomSpline.position()` / `tangent()` signatures used identically in `Spline.test.ts`, `Spline.ts`, and `RailController.ts`.
- `ClipAxis` Literal consistent between `DissectionStore`, `ClippingPlane`, and `Manifest.objects[].preferred_clip_axis`.
- Modal function names (`scene_gen_hunyuan`, `scene_gen_stock`, `object_gen`) match between `modal/*.py` and `backend/app/modal_client/*.py`.

No inconsistencies.

Execution handoff

Plan complete and saved to `docs/superpowers/plans/2026-05-14-interactive-museums-plan.md`.
Two execution options:

1. **Subagent-Driven (recommended)** — fresh subagent per task, two-stage review between tasks, fast iteration. Required sub-skill: `superpowers:subagent-driven-development`.
2. **Inline Execution** — execute in this session using `superpowers:executing-plans`, batch with checkpoints.

The user has pre-approved "auto mode" → **subagent-driven-development**.